

예산·결산 심사 업무의 AI 에이전트 설계와 활용

답만 하는 AI를 넘어, 계획·조사·집필·검증까지 하는 AI를 내 업무에 들이는 법

박 주 이

한양대학교 과학기술정책연구소 연구교수 · parkjui92@gmail.com

2026. 9. 16.(수)

혹시 이렇게 쓰고 있지는 않은가?

- ☐ "완료했습니다"라는 말을 그대로 믿고 쓴다
- ☐ 오류가 나면 "안 돼, 고쳐줘"라고만 말한다
- ☐ 잘 안 풀리면 새 대화를 열고 처음부터 다시 설명한다
- ☐ 결과가 나빠지면 같은 요구를 반복한다 — 실은 맥락이 소진된 상태
- ☐ 한 번의 지시로 산출물을 받는다 — 그럴듯하지만 알맹이가 없다
- ☐ 몇 번 써 보고 성능 부족이라 결론 내고 멈췄다

0 몇 개나 해당하나요?

원리를 이해하고, 내 것을 만든다

1부 교육 — 개념 · 시연 · 해부

1절 왜 지금인가 — 검토보고서 쓰는 일은 어디까지 바뀌나

2절 에이전트란 무엇인가 — 부품과 루프

3절 ▶시연 — 실제 운용 팀과 예결위 업무 적용

4절 공개된 킷으로 부품 뜯어보기

5절 틀리면 안 되는 것을 규칙으로 굳힌다

2부 실습 — 스킬 · 킷 · 내 에이전트

준비 설치/셋팅 · 작업 폴더 · 전역 설치 개념

실습1 교정 스킬(paper-proofread) 설치·실행

실습2 보고서 쓰는 AI 팀(킷) 설치 — 브리프 가동

실습3 내 업무용 에이전트 기획안 작성과 바이브코딩

INSIGHT

오늘 가져갈 것 — **원리**(에이전트·스킬을 읽고 고치는 눈) · **규칙**(틀리면 안 되는 것을 강제하는 법) · **내 것**(담당 업무용 개조판)

SECTION 1

왜 지금인가

보고서 쓰는 일은 어디까지 바뀌나 — 기대와 불신 사이

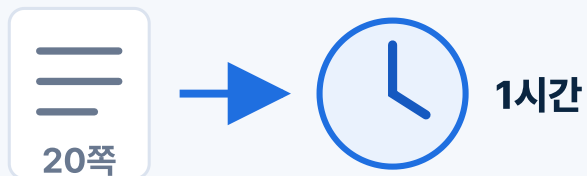
01

'단순 챗봇 시대'의 종언

묻고 답하는 챗봇에서 스스로 일하는 에이전트로

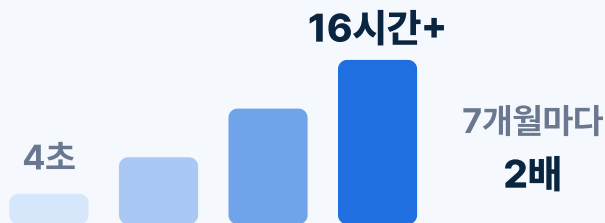
🚄 연구 현장이 재편된다

Brookings — "기차는 이미 떠났다".
에이전틱 AI로 **20쪽 보고서를 1시간 내 산출**
Messing & Tucker, 2026.3



📈 해내는 일의 길이가 폭증

METR — 자율로 완수하는 과업 길이 **4초**
(2019) → 16시간+(2026) · 약
7개월마다 2배
Measuring AI Ability to Complete Long
Tasks, 2025.3



🕒 패러다임이 재정의된다

Anthropic — 목표를 주면 스스로 **계획·**
도구 사용·검증을 반복하는 시스템으로
Building Effective Agents, 2024.12

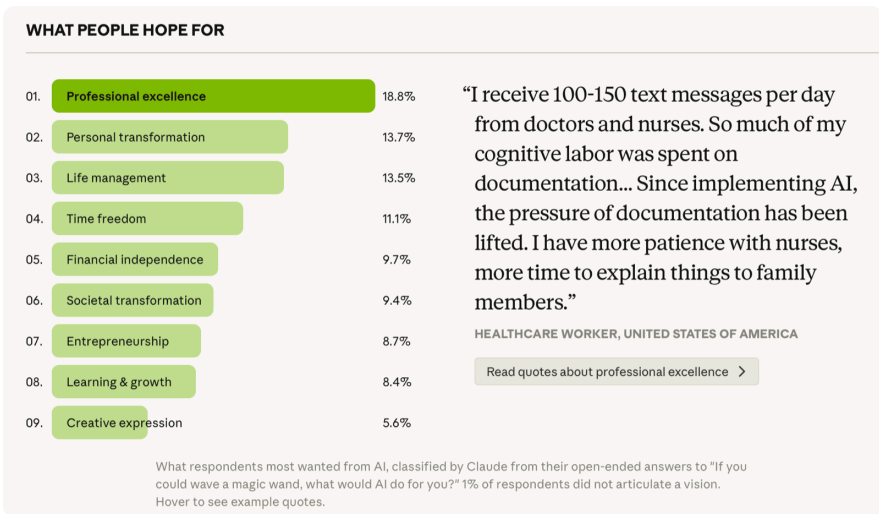


용어 에이전틱 AI — 목표를 주면 스스로 계획을 세우고 도구를 골라 실행하며, 결과를 보고 스스로 고치는 AI

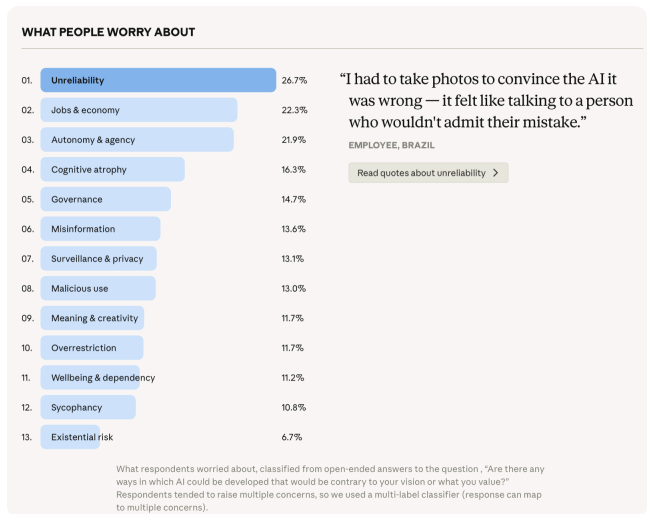
출처 Brookings 「The train has left the station」(2026.3) · METR 「Measuring AI Ability to Complete Long Tasks」(2025.3) · Anthropic 「Building Effective Agents」(2024.12)

사람들은 AI에 일을 맡기고 싶어 하면서도, 신뢰를 가장 걱정한다

역대 최대 규모의 질적 연구 — AI 인터뷰어가 159개국 80,508명과 심층 인터뷰한 결과



기대 1위 — 전문성 향상 18.8% · 루틴은 위임하고 전략 업무에 집중 (개인적 성장 13.7 · 생활 관리 13.5 · 시간적 자유 11.1%)



우려 1위 — 비신뢰성(Unreliability) 26.7% · 환각·부정확 출력·검증 부담 (고용·경제 22.3 · 자율성 상실 21.9 · 인지 퇴화 16.3%)

INSIGHT

우려 1위가 비신뢰성(26.7%) — 숫자가 생명인 예결산 보고서에서 이 긴장이 가장 크다

가장 무서운 오류는 '그럴듯한' 오류다

같은 요청으로 받은 AI 초안을 제3자가 **인용 URL 전수 열람**으로 감사한 실측 기록

- ◆ "경험률 **67.8%**" — 숫자는 실재하지만 **전혀 다른 지표**의 값이었다
지표 혼동 — 지어낸 숫자보다 검산이 어렵다
- ◆ "불편률 64.2%" — 인용된 연구보고서 **원문 237쪽 어디에도 없는** 수치
문서 귀속 오류 — 수치 자체는 실재하나 실제 출처는 다른 문서
- ◆ 문서 말미엔 **진짜 URL 10개** — 전체가 검증된 것처럼 보이는 착시
"인용 10건, 원문 열람 0회"가 만드는 위험

CAUTION

지적받는 것은 AI가 아니라 **보고서에 이름을 올린 사람**이다

그래서 오늘 — '검증까지 하는 AI 팀'을 만드는 법을 배운다

프롬프트 문장술이 아니라 — **판을 짜는 법**을 배운다

2절 · 개괄

에이전트란 무엇인가
챗봇과 다른 점 · 부품과 자율
루프



3절 · 예시·시연

실제 팀을 돌려 본다
동향 모니터링 · 결산 검토보고서



4절 · 부품 해부

공개 킷을 뜯는다
CLAUDE.md·스칼·API·MCP



5절 · 규칙

틀리면 안 되는 것을 강제
급소 목록 · 검산 게이트



2부 · 실습

내 업무용 AI 팀 초안

이 교육이 아닌 것

프롬프트 요령 · 최신 기능 소개 · 도구
사용 매뉴얼
1년이면 낡는다

INSIGHT

챗봇 활용팁이 아니라 **업무 시스템 설계**의 시간 — 도구가 바뀌어도 남는 원리에 집중한다

이 교육인 것

일을 **쪼개 위임하고 · 검증을 공정**에 박고 ·
자산으로 남기는 설계 원리
도구가 바뀌어도 남는다

오늘의 약속

끝나면 **내 업무용 AI 팀 초안** 하나를 들고
나간다 — 완성이 아니라 **돌아가는 뼈대**
2부 실습 산출물

SECTION 2

에이전트란 무엇인가

챗봇과 무엇이 다른가 · 부품과 자율 루프

02

챗봇과 에이전트는 어떻게 다를까

"답변해 주는 비서" vs "실행해 주는 동료"

구분	일반 챗봇 (웹 대화)	AI 에이전트
응답 방식	한 번 묻고 한 번 답	자율 루프 — 목표를 주면 판단·실행을 반복해 결과를 가져옴
파일 접근	하나씩 업로드	내 PC의 폴더·문서를 직접 읽고 씀
실행	답변 생성까지	실제 작업 수행 — 표 만들고, 저장하고, 변환
외부 도구	제한적	MCP로 통계·법령·의안 시스템 연동
기억	대화 끝나면 리셋	규칙·맥락을 파일로 유지
재사용	매번 다시 설명	절차를 스킬로 박제 해 한 마디로 재실행

용어 **에이전트** — 스스로 계획을 세우고 도구를 골라 쓰며, 결과를 보고 스스로 고치는 AI. 이 둘이 없으면 아직 챗봇이다

에이전트 = 두뇌 · 손발 · 기억 · 자율 루프의 조립품

"목표만 주면, 알아서 도구를 골라 써서 끝까지 해내는 AI" — 챗봇은 대답까지, 에이전트는 일 처리까지

LLM — 두뇌

언어를 이해하고 다단계 계획을 세우고 판단한다

역할: 무엇을 어떤 순서로 할지 결정

도구 — 손발

파일·통계·법령·문서 시스템과 연동해 **실제로 행동**한다

통로: MCP · API (뒤에서 상세)

메모리 — 기억

대화 맥락과 규칙을 유지한다 — 우리 과의 작성 규칙을 계속 기억

실체: CLAUDE.md 등 파일

추론 루프 — 자율성

스스로 판단하고 반복 실행한다 — **될 때까지**

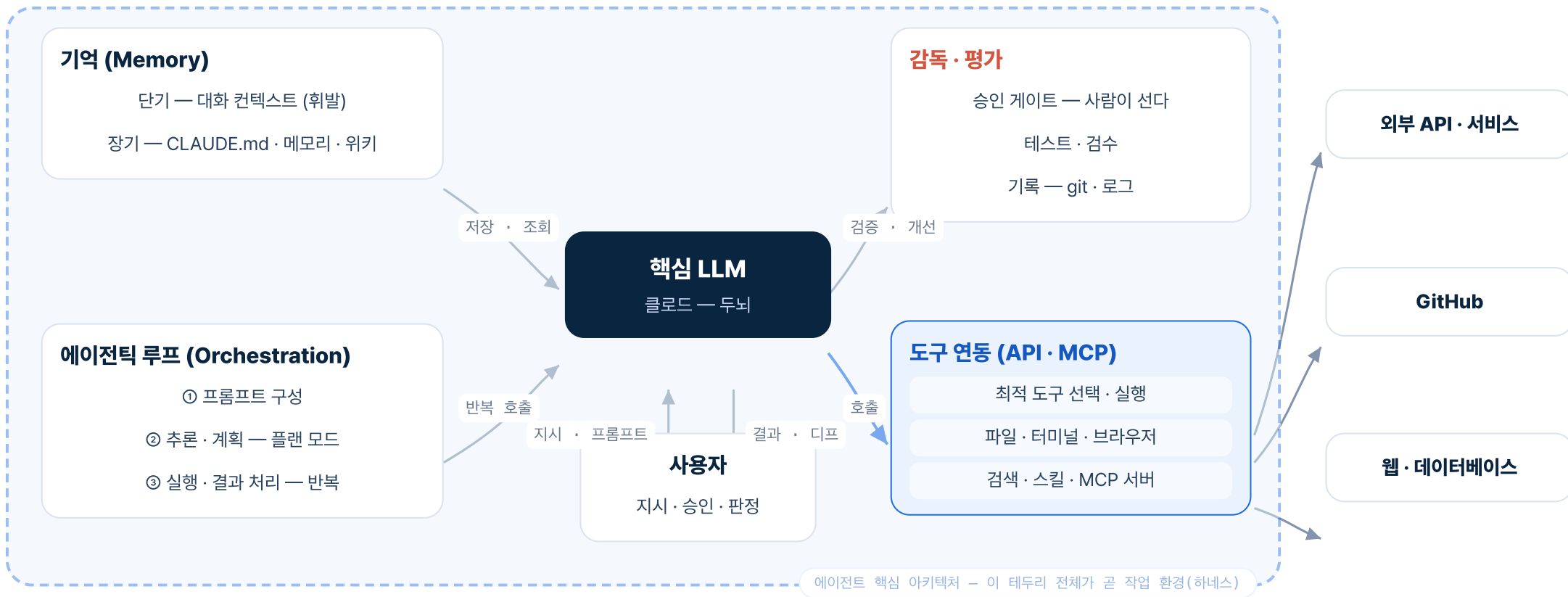
챗봇과의 결정적 차이

용어 LLM(대형언어모델) — 방대한 텍스트로 학습해 언어를 이해·생성하는 AI. 에이전트의 판단 엔진

출처 Anthropic, 「Building Effective Agents」(2024.12) — 목표지향 · 도구 사용 · 자율 반복의 에이전트 정의 anthropic.com/engineering

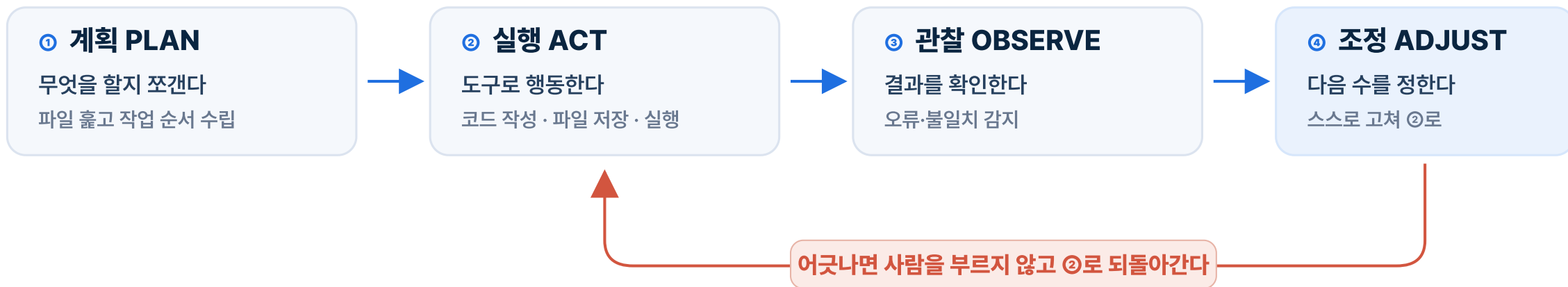
에이전트 하나를 열어 보면 — 부품 배치도

가운데 두뇌를 루프가 돌리고, 좌우로 **기억 · 도구 · 감독**이 붙는다



에이전트는 계획→실행→관찰→조정을 스스로 돈다

챗봇은 ②에서 멈춘다 — 에이전트는 ③에서 어긋나면 ④를 거쳐 ②로 되돌아간다



사람이 하는 일 — 목표를 준다 · 결과를 검토한다 · 필요하면 중간에 끼어든다

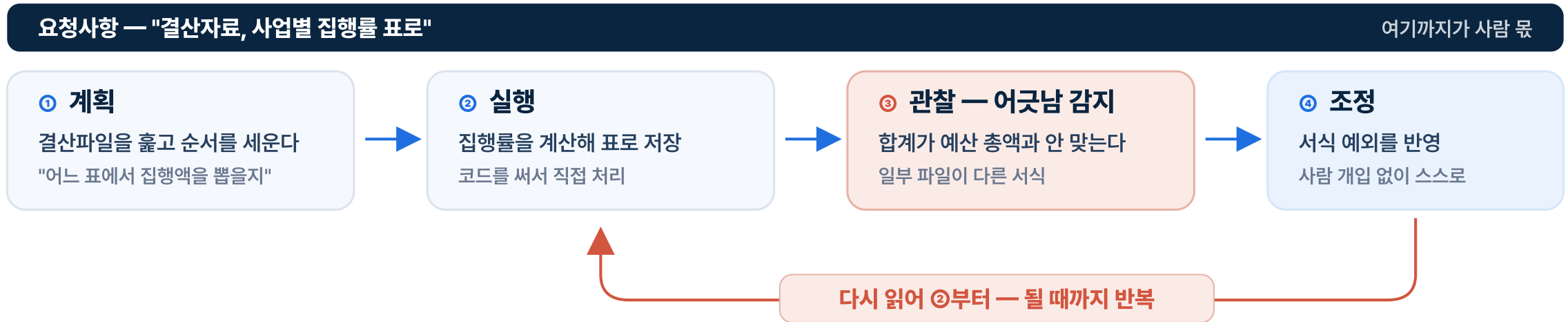
나머지 한 바퀴는 알아서 돈다

INSIGHT

사람 개입 없이 "에러 나면 고치고 다시 실행"을 반복한다 — 이것이 8초 답변과 45분 자율 작업의 차이다

[예를 들면] 루프가 도는 장면 — "결산자료, 사업별 집행률 표로"

핵심은 ③에서 어긋나도 ④가 스스로 고쳐 ②로 되돌아간다는 것



한 줄 말하고 자리를 비워도 표가 나와 있다 — 챗봇이라면 ②에서 멈춰 "왜 안 맞지?"를 사람이 찾는다

INSIGHT

사람은 "이 자료 사업별로" 한 줄만 말했다 — 나머지 계획·실행·오류수정은 에이전트가 돌렸다 — 이것이 챗봇과의 결정적 차이다

왜 클로드 코드인가

주요 도구는 기능이 이미 **서로 수렴**했다 — 고르는 기준은 우열이 아니라 **내 업무 환경에 맞는가**

내 자료를 직접 다룬다

내 폴더의 **한글 · 엑셀 · PDF**를 **읽고 · 고치고 · 만든다**

검색하고 · 파일 열고 · 계산하고 · 정리해 온다 — 부처 제출자료가 그대로 재료가 된다

기획부터 산출물까지 한 흐름

조사·기획 → 만들가·고치기 → **내보내기까지** 한 자리에서

자료가 바뀌면 다시 돌려 같은 산출물을 새로 뽑는다 — 중간 산출물이 파일로 남는다

규칙을 파일로 못 박는다

업무지침도 절차서도 **md 한 장** — 읽고 고치고 복사한다

과 단위로 나누면 그대로 과의 표준 — 5절 재정 규칙이 여기 들어간다

팀으로 쪼개고 검증을 박는다

역할별 팀원과 **독립 검증자**를 공정 안에 둔다

초안을 쓴 쪽이 스스로 통과시키지 못하게 하는 구조

INSIGHT

"코드"라는 이름에 겁먹지 않아도 된다 — **코드는 수단이고 산출물은 업무 결과물** · 도구가 바뀌어도 남는 것은 판을 짜는 법

비고 터미널을 열지 않고 **앱에서 한국어로** 쓴다 — 바뀐 파일이 화면에 남아 검증을 눈으로 한다 · Copilot · Cursor · Codex도 자율 실행 · MCP · 규칙 파일 · 서브에이전트를 지원한다 — 우열이 아니라 오늘 실습 환경이 클로드 코드일 뿐

채팅 · 코워크 · 코드 — 어디서 여느냐가 다르다

같은 모델·같은 개념이 세 자리에 다 붙는다 — 다른 것은 **내 컴퓨터를 어디까지 건드리느냐**

채팅 Chat 주고받으며 생각하는 자리 파일은 올리고 내려받는다 — 내 폴더에 저장되지는 않는다	코워크 Cowork 맡겨 두는 자리 — 내 폴더를 읽고 같은 자리에 저장 긴 작업을 던져 두고 결과만 받는다	코드 Code — 오늘 폴더를 열어 읽고 고치고 만든다 권한 창 · 바뀐 파일 · 폴더가 한 화면 — 검증을 눈으로 한다
---	--	---

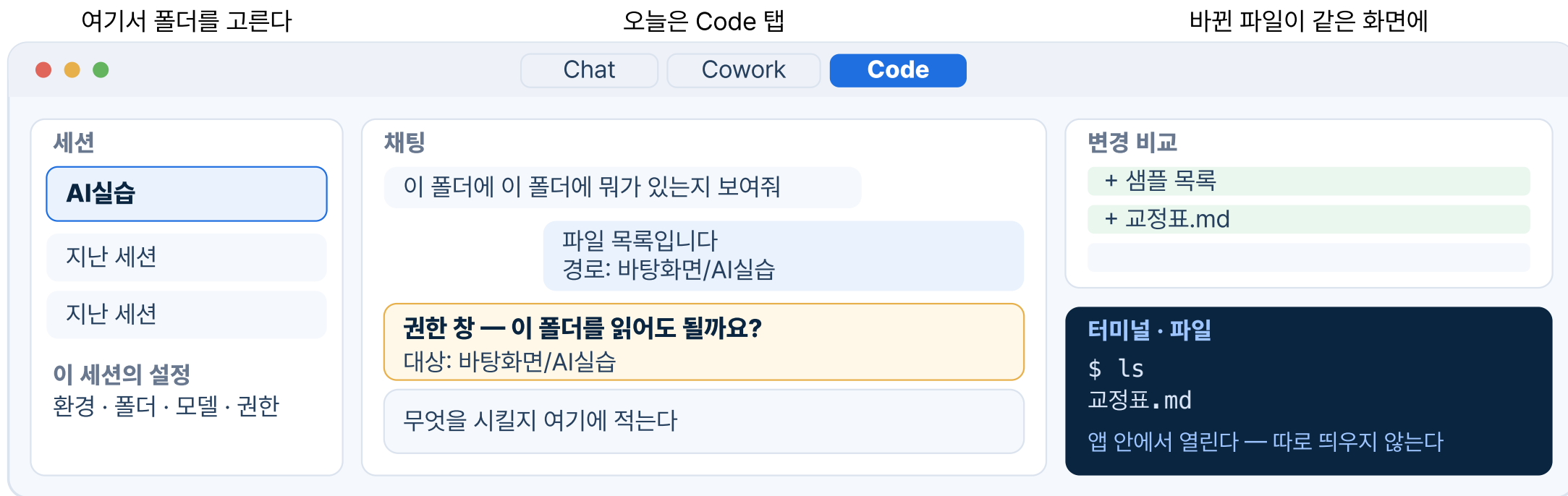
부품	코드 탭 — 오늘	채팅 · 코워크
스킬	<code>~/.claude/skills/</code> 에 파일 로 둔다	설정에서 켜거나 압축 파일로 올린다 — 계정으로 동기화
MCP	<code>/mcp</code> 로 붙이고 확인한다	커넥터 로 붙인다 — 로컬 연결은 앱을 거친다
업무 규칙	폴더의 <code>CLAUDE.md</code>	같은 파일은 없다 — 프로젝트 지침 이 그 자리

INSIGHT

오늘 코드 탭을 쓰는 이유 — **무엇을 건드렸는지가 화면에 남는다**

클로드 코드를 처음 커면 이렇게 생겼다

터미널에 코드 명령을 치지 않는다 — 데스크톱 앱에서 한국어로 같은 일을 한다



Claude 데스크톱 앱 Code 탭 — 강의용 구성도(배치는 앱 버전에 따라 다를 수 있다)

INSIGHT

가운데 **노란 권한 창**이 가장 자주 보게 될 것 — 무엇을 건드리는지 **대상 경로**를 읽고 허용한다

SECTION 3 · LIVE

무슨 일을 맡길 수 있나

매주 도는 동향 팀 · 결산 검토보고 · 시정요구 전수 대조 — 실제로 도는 장면

개발자의 전유물이던 일이 자연어 한 줄이 됐다

작업	이전 — 개발자의 영역	지금 — 자연어 한 줄
파일 자동 처리	Python 스크립트 직접 작성 → 터미널 실행 → 에러마다 수정 → 재실행	"이 폴더의 보고서 전부 읽고 주제별 요약표 만들어줘"
데이터 연동	API 연동 코드 · 인증 처리 · 예외 처리 지식 필수	"열린재정에서 부처별 집행을 받아서 표로 정리해줘"
통계 분석	라이브러리 설치(버전 충돌) · 검정 코드 · 서식 수작업	"연도별 추이 분석하고 보고서 서식 으로 저장해줘"
도구 배포	서버 · 도메인 · 프레임워크 설정	"점검 결과를 매주 알림 으로 보내는 도구 만들어줘"

INSIGHT

공통점 — "코드 작성 → 실행 → 에러 수정 → 재실행" 루프를 **AI가 대신 돈다** — 사람은 요구와 검수만

매주 하는 동향조사, 병목은 수집이 아니라 검증이다

챗봇은 모으는 데까지 — 깨어나고 · 의심하고 · 내보내는 자리는 에이전트라야 돈다



조사자들은 병렬로 뛰고, 검증자는 독립적으로 의심한다

조직의 상호검토를 그대로 옮긴 구조 — 사람은 맨 위 한 명에게만 말한다



실제 에이전트 활용 사례

정형화된 절차 + 명확한 산출물이 있으면 **거의 다 위임할 수 있다**

★ 글로벌 동향 주간 브리핑 팀

조사자 3인 + 독립 검증자가 매주 자동으로
국내외 정책·담론·기술 동향을 브리핑
앞 장에서 구조를 본 그 팀

★ 정책연구보고서 작성 팀

설계 → 조사 → 집필 → 검수 → 한글
(.hwp)까지 5인 팀이 완주
오늘 공개 배포하는 플러그인

★ 재정수반법안 선별

계류 의안 수집 → 재정 신호 감지 →
비용추계서 첨부 점검 → 우선순위
국회 업무용 맞춤 스킴

예·결산 분석 보조

사업설명자료 대량 요약 → 쟁점 후보 도출
→ 예산 현황표 → 전년 지적사항 반영 추적
2부 실습에서 여러분이 만들 개조판

부처 제출자료 대량 파싱

HWP·PDF 수십 건을 표·데이터로 구조화 —
수치 대사의 앞단
kordoc

자료·문헌 대량 요약

보고서·논문 수백 편을 일괄로 읽어 주제별
분류·요약표 생성

INSIGHT

공통점 — 반복되고 · 절차가 뚜렷하고 · 산출물이 정해진 일 — 특별한 업무가 아니라 **가장 자주 하는 업무**가 위임 대상이다

[사례] 재정·의회 업무에서 이미 돌고 있는 것들

연구실이 아니라 공무원이 자기 업무에 만들어 쓰는 도구들

무엇	어떻게 돌아가나
집행대장 공직 개인 개발	지방재정365 공개 API로 전국 243개 지자체 세부사업 46만 건의 집행률 을 매일 아침 정리해 올린다
지자체 보건복지 예산 분석	지방재정365 세부사업 단위 로 사회복지·보건 예산과 사업을 갈라 본다
학교예산 시각화	에듀파인 엑셀을 읽어 표로만 보던 학교회계를 그림으로 바꾼다
경기도의회 주간업무보고	22개 부서의 보고를 취합·검토·조판 까지 한 흐름으로 묶은 시제품
회의록 작성 도우미	회의 기록이 공개되기까지 걸리는 시간 을 줄이려고 만든 도구

INSIGHT

우리가 오늘 만들려는 것과 **같은 계열의 물건이 이미 여럿 돌고 있다** — 첫 사례는 예결위 업무의 지방재정판이다

[사례] 제약이 많은 업무에서도 길이 있다

검토보고서와 가장 가까운 쪽 — 법령을 대조하고, 감사 의견을 쓰고, 망 밖으로 자료를 내보내지 않는다

무엇	어떻게 돌아가나
법령 체계 분석 과학기술혁신본부	법령 MCP로 법령 간 근거·위임·연계·충돌을 정리했다 — 조문 대조를 기계에 맡긴 실제 사례
LexDiff	법령·조례·판례·해석례를 통합 비교·검색 — 다른 의회가 그대로 재사용해 자기 시스템으로 세웠다
감사 의견서 자동화	일상감사 처리대장을 관리하고 검토 의견서 초안을 로컬 AI로 쓴다 — 감사 자료를 밖으로 내보내지 않는다
망분리 로컬 에이전트	모델까지 노트북에서 돌려 인터넷 없이 국가통계포털을 조회하는 시연
KOSIS 자연어 조회	국가통계포털 통계를 말로 물어 표로 받는다

INSIGHT

보안이 까다로운 업무일수록 어디까지를 밖에 두고 어디부터를 안에 둘지가 설계의 핵심이 된다

검토보고서 계열 — 해마다 같은 모양으로 되풀이된다

예산안 한 번, 결산 한 번, 추경 두세 번 — 한 해에 네댓 사이클

산출물	말길 수 있는 일	지키는 장치
결산 검토보고서	집행률·이월·불용 표 생성 → 집행 부진·연례 반복 사업 선별	검산 게이트 합계 · 단위 · 분모
예산안 검토보고서	사업설명자료 요약 → 증감 사유 대조 → 쟁점 후보 도출	원자료 대조 열린재정 편성·집행
예비비·기금 검토보고	사용 실적과 당초 요건 대조 → 요건 충족 여부의 논점	법령 조문 대조 국가재정법 · 헌법 §55②

INSIGHT

양식이 고정돼 있고 자료원이 정해져 있다 — 쪼개어 말기기에 이보다 나은 조건은 없다

심사 운영 산출물 — 사이클마다 한 벌씩 다시 만든다

보고서만이 아니다 — 심사를 굴리는 문서가 따로 한 벌 있다

산출물	말길 수 있는 일	지키는 장치
시정요구 조치결과 점검	전년도 시정요구를 전수 대조 해 자기보고와의 어긋남을 목록으로	문안 원문 보존 요약 금지 · 유형 병기
서면질의·요구자료	양식 생성 → 회신 취합·정렬 → 미회신 표시 → 답변 간 상충 탐지	답변 일자 병기 주장과 사실을 구분
소위 심사자료	심사순서·부처별 쟁점 요약을 매일 갱신 되는 형태로 재생성	전일 대비 변경분 표시

INSIGHT

보고서 사이를 메우는 문서가 사이클마다 다시 만들어진다 — 양식이 고정된 만큼 **가장 먼저 말길 수 있는 쪽**이다

게시판에 쌓인 것들 — 그대로 맡길 수 있는 일

앞 장의 목록은 짐작이 아니다 — 정책자료 게시판에 쌓인 실물

보도자료

자료실

정책자료

공청회/간담회/세미나

(인사)행문회

사진/동영상

양식/서식

기타

특위

정책자료

전체

업무편람

소관기관업무보고

활동계획결과보고

정책현안자료

의안/규칙

예산-결산 기금

정책연구용역보고서

시정세정보고서

기타

제목

제목을 입력해주세요.

일자

제[2024-07-09]

제[2024-07-09]

전체 1,246 건 (1/125 페이지) 10 목록

검색

목록다운로드

번호	위원회	제목	작성일	구분	미리보기	첨부파일	조회
1246	예산결산특별위원회	2025회계연도 결산소위 심사순서 (0908)	2026-09-07	기타			18
1245	예산결산특별위원회	2025회계연도 결산소위 심사순서 (0904)	2026-09-03	기타			20
1244	예산결산특별위원회	2025회계연도 결산소위 심사순서 (0903)	2026-09-03	기타			22
1243	예산결산특별위원회	2025회계연도 결산소위 심사순서 (0902)	2026-09-01	기타			39
1242	예산결산특별위원회	2025회계연도 결산심사소위원회 확대배치도 ...	2026-09-01	기타			45
1241	예산결산특별위원회	2027년도 예산안 예비결목 및 공통요구자료 II...	2026-08-27	예산-결산 기금			125
1240	예산결산특별위원회	2025회계연도 결산심사 전체회의 기획조정실...	2026-08-20	기타			97

1/125

예결위 정책자료 게시판 — 2026-09-13 캡처

- ◆ 결산소위 심사순서가 **날마다 새로 올라온다**
- ◆ 좌석배치도 · 부처 협조사항 · 공통요구자료도 사이클마다 되풀이
양식은 그대로, 연도와 대상만 바뀐다
- ◆ 이런 것이 **쫄개어 맡기기 가장 쉬운 일**

INSIGHT

양식이 고정된 문서가 **해마다 같은 자리에서 다시 만들어진다** — 자동화의 조건이 이미 갖춰져 있다

결산 검토보고서 한 항목이 만들어지는 과정

앞의 동향 팀과 구조는 같고 배역만 다르다 — 검토보고 항목의 짜임을 그대로 옮긴 것



INSIGHT

기계가 **표와 후보를 깔아 놓으면**, 사람은 남은 시간을 전부 **판단**에 쓴다 — 지적할 값어치가 있는가, 어느 세기로 쓸 것인가

정부가 조치완료라고 보고한 것을 다시 세어 본다

예결위가 이미 하고 있는 일 가운데 **기계가 가장 잘하는 일**이 여기 있다 — 전수 대조

2025회계연도 결산 시정요구

1,987건

제도개선 1,493 · 주의 437 · 시정 148



정부 자기보고

조치완료 1,603건

= 80.7%

조치중



국회가 다시 세어 본 결과

완료 보고 가운데 실제로는 미흡 12건

그중 5건은 해마다 되풀이되는 미조치 · 조치중 항목에서도 노력 미흡 2건

◆ 국회는 자기보고를 그대로 믿지 않는다 — 1,987건을 맞대어 본다

성실함이 아니라 분량의 문제 — 사람이 다 볼 양이 아니다

◆ 기계가 후보를 좁히고, 사람이 그것만 원문으로 확인한다

◆ **AI에 넘기면 안 되는 것** — '조치완료'를 인정할지의 판단

문구가 비슷하다고 이행이 같지 않다 — 제도 변경으로 확인한다

INSIGHT

에이전트를 감독하는 방식이 국회가 정부를 감독하는 방식과 같다 — 보고를 받되, 근거로 대조한다

SECTION 4

부품을 뜯어본다

CLAUDE.md · 스킬 · 서브에이전트 · 오케스트레이터 · MCP · API · 에이전트팀 — 실물로 배운다

04

공개해 둔 킷 — 보고서를 끝까지 쓰는 AI 팀

보고서 써줘 한 줄이면 팀이 돈다 — 사람이 서는 자리는 ★목차 승인 한 번



- ◆ **관문①** — 잘못된 목차로 가는 **비싼 헛작업**을 집필 전에 차단 · **관문②** — 논리 · 근거 · 타당성 · 정량성 · **출처 실재**
- ◆ **★목차 승인** — 자동화 한가운데 박아 둔 **사람의 자리** · 기관의 결재·검토 라인을 공정으로 옮긴 것
- ◆ 산출은 표준 보고서(본문 50쪽+)와 **단형 정책브리프(4~8쪽)**

2교시 실습은 시간 안에 완주되는 브리프 모드

github.com/parkjui92/

← 2교시에 이 주소로 설치한다

오케스트레이터와 에이전트 팀

역할 (agents/*.md)	맡는 일	예시 프롬프트
조율자 orchestrator ★ 말을 거는 유일한 창구	한 마디를 받아 팀원을 순서대로 호출·조율	지방재정 건전성 정책연구보고서 써줘
설계자 designer	의도 분석 · 연구질문 · 목차 설계	지방재정 건전성 연구의 목차를 설계해줘
조사관 investigator	통계·문헌·사례 조사 — 모든 사실에 출처 병기	지방채 발행 추이를 출처와 함께 조사해줘
작성자 writer	논증형 본문 집필 · 참고문헌 정리	3장 현황분석 본문을 작성해줘
검수자 reviewer ★	설계 관문 + 초안 검수 — 독립 검증자	초안을 검수하고 수정요청서를 작성해줘
변환가 hwp-x-exporter	한글(.hwp) 변환 · 표 개수 자체 검증	최종본을 한글(.hwp) 파일로 변환해줘

INSIGHT

사람은 **맨 윗줄에만** 말한다 — 아래 다섯은 조율자가 순서대로 부른다 · 검수자만 **설계와 초안에 두 번** 등판한다

검증자는 무엇을 보는가 — 챗봇과 갈리는 자리

같은 문장을 놓고 **챗봇은 읽고 넘어가고** — 검증자는 **열어서 대조한다**

들어온 한 줄 — "OO 사업 경험률은 67.8%로 나타났다(□□연구원, 2025)"

① 출처 등급을 매긴다

1 정부·기관 공식 · 2 국책연구·학술지

3 주요 언론 · 4 블로그·SNS

4등급은 그것만으로 근거가 되지 않는다
등급이 낮으면 다른 출처를 하나 더 요구한다



② 서로 맞는지 대조한다

수치 · 날짜 · 인명 · 기관명을 맞대기

같은 보도자료를 받아쓴 기사끼리는

교차검증이 아니다 — 재인용 사슬

출처가 여럿이어도 뿌리가 하나면 한 건으로 센다



③ 실제로 열어 본다

그 주소가 열리는가

원문에 그 수치가 있는가

그 문서가 맞는가

1절에서 본 오류가 여기서 걸린다



챗봇이라면

문장이 매끄러우니 그대로 통과시킨다

인용이 붙어 있으면 확인된 것처럼 읽힌다

검증자라면

매끄러울수록 더 의심한다

확인 못 한 것은 빼고 — 뺄 흔적을 남긴다

이 부품만 알면 나만의 AI 팀을 조립할 수 있다

Claude Code를 이루는 핵심 부품 — 예시는 전부 **예결위 업무**로 바꿔 읽는다

부품	역할	예결위라면
CLAUDE.md	항상 읽히는 업무 매뉴얼 — 역할·규칙·금지사항	"우리 과 보고서 작성 규칙 — 단위는 억원 · 원자료 대조 필수 · 중립 서술"
스킬 Skill	불러 쓰는 작업 절차서 (SOP) — 반복 작업의 표준	"시정요구 조치결과 대조 절차"를 한 마디로 가동
슬래시 명령어 Slash Command	자주 쓰는 절차의 단축키	/조사분석관 → 수집→정리→표 생성
서브에이전트	독립 컨텍스트의 팀원 — 병렬·전문화	자료조사 담당 · 수치검증 담당 을 따로 고용
훅 Hook	정해진 시점의 자동 규칙 — 잊어도 지켜짐	저장할 때마다 수치 표기 규칙 자동 점검 · 원본 수정 차단

틀 CLAUDE.md + 스킬 + 명령어 + 서브에이전트 + 훅 = **나만의 에이전트** — 앞 절 시연의 팀과 이 절의 킷이 그 실물이다

CLAUDE.md — 매번 반복하던 것을 규칙으로

에이전트에게 주는 "업무 매뉴얼" — 한 번 적어 두면 모든 대화에서 자동으로 읽힌다

없으면	있으면
매 대화마다 처음부터 다시 설명	한 번 적으면 계속 유지
사람마다 달라 품질이 들쭉날쭉	과 전체가 같은 표준
새 대화를 열면 맥락이 초기화	맥락이 영속된다

정책연구보고서 작성 규칙 (이 폴더의 모든 작업에 적용)

- 분석틀** 승인된 목차와 분석틀을 따른다 — 구조를 임의로 바꾸지 않는다
- 논증** 단락마다 무엇이 문제인가 → 왜 그런가 → 그래서 무엇을 해야 하는가
- 대안** 단일안을 단정하지 않는다 — **효과성·실현가능성·비용·수용성**으로 비교한 뒤 권고안과 근거를 제시한다
- 정량** "개선"이 아니라 "**X에서 Y로 ($\Delta Z\%$)**" — 근거가 없으면 **[보강 필요]**로 표시하고 **지어내지 않는다**
- 출처** 인용한 사실에 (출처: ...)를 단다 — 기준연도와 정의를 본문에 반영
- 문체** 결정자가 5분에 핵심을 잡도록 **두괄식** — 현학적 표현을 피한다
- 제언** **수단·주체·재원·일정·기대효과**까지 적는다
- 금지** 미공개 자료를 넣지 않는다 · 출처 없는 수치를 쓰지 않는다

INSIGHT

한마디로 — 신입에게 주는 온보딩 문서

위치 파일 하나(./CLAUDE.md)를 작업 폴더에 두면 끝 — 오른쪽은 **오늘 배포하는 정책연구 킷**의 집필 규율을 작업 폴더 규칙으로 옮긴 것 · 예결위판(재정 수치 규칙)은 5절에서 본다

에이전트는 코드가 아니라 업무기술서 — md 파일 한 장이다

policy-research-kit을 열면 — 안에 있는 것은 **전부 사람이 읽는 파일**이다

```
policy-research-kit/
├── agents/                ← 팀원 다섯의 업무기술서
│   ├── policy-research-designer.md    설계자
│   ├── policy-research-investigator.md  조사관
│   ├── policy-report-writer.md         작성자
│   ├── policy-report-reviewer.md       검수자
│   └── hwp-exporter.md                변환가
├── skills/                ← 절차서 여섯
│   ├── 조율 · 설계 · 조사 · 집필 · 검수 · 한글 변환
├── examples/             ← 전 과정 산출물 실물 두 벌
├── docs/                 ← 설계 문서 · 순정 대비 실측 기록
└── README.md             ← 설치와 사용법
```

- ◆ **md 파일 수 = 팀원 수** — 역할·절차·금지사항이 문장으로 적혀 있다
- ◆ 사람에게 하는 **인수인계**가 곧 에이전트 정의
- ◆ 파일이라 **복사 · 수정 · 버전 관리**가 전부 된다
- ◆ 블랙박스 아니다 — **검수 기준·조사 규칙까지 열어 고친다**

INSIGHT

예결위라면 — 같은 자리에 **검토보고서 목차**와 **급소 검수 기준**을 넣으면 그대로 우리 팀이 된다

시연 여기서 파일 하나(`policy-report-reviewer.md`)를 라이브로 열어 본다 — 다음 장이 그 파일이다

그 파일을 열어 본다 — 기호 몇 개가 전부다

킷의 **검수자** 파일 — 특별한 문법이 아니라 제목 · 목록 · 굵게 세 가지로 되어 있다

```
agents/policy-report-reviewer.md - 실물 발췌

# 정책연구 검토관 - 두 관문을 지키는 사람 ← # 은 제목

## 모드 2 - 초안 검수 (집필 후) ← ## 은 하위 제목
1. **논리 정합성** - 현황→쟁점→대안→제언이 이어지는가 ← 1. 번호 목록
2. **근거 충실성** - 사실 주장에 출처가 있는가
3. **정책 타당성** - 수단·재원·주체가 현실적인가
4. **정량성·표현** - 수치가 맥락과 함께 제시됐는가
5. **참고문헌·출처 검증** - 본문 인용과 1:1 대응인가

## 작업 원칙
- **위치·문제·해결** - 어디가 / 왜 문제이며 / 어떻게 고치라 ← - 은 글머리표
- **특점 아닌 진실** - 듣기 좋은 말보다 옳은 지적을 ← ** ** 는 굵게
```

기호	뜻
# · ##	제목 · 하위 제목
- · 1	글머리 · 번호 목록
굵게	강조
`코드`	명령 · 파일명

INSIGHT

예결위라면 — 이 다섯 축 아래에 **회계연도 · 단위 · 분모 · 검산** 한 줄을 더 적으면 우리 검수 기준이 된다

스킬 — 반복하는 지시를 절차서로 굳힌다

매번 길게 설명하던 절차를 **파일 하나로** — 다음부터는 **이름만 부르면** 그대로 실행된다

`policy-research-kit/skills/`

<code>└─ rnd-policy-research-orchestrator</code>	조율 — 팀을 순서대로 부른다
<code>└─ policy-research-design</code>	설계 — 연구질문 · 분석틀 · 목차
<code>└─ policy-research</code>	조사 — 출처 병기 근거 수집
<code>└─ policy-report-writing</code>	집필 — 논증형 본문
<code>└─ policy-report-review</code>	검수 — 설계 게이트 + 초안 검수
<code>└─ rnd-hwpx-export</code>	변환 — 한글(.hwpx) 생성

- ◆ **스킬 없이**는 매번 다시 설명한다
그날그날 품질이 다르고 빠뜨리는 항목이 생긴다
- ◆ **스킬이 있으면** 누가 불러도 같은 절차
한 번 만들면 과 전체가 공유한다
- ◆ 절차가 글이라 **읽고 고쳐 내 것으로** 만든다

INSIGHT

예결위라면 — **시정요구 조치결과 대조 · 집행 부진 사업 선별 · 서면질의 회신 취합**이 좋은 후보

내 업무도 스킬이 된다 — '시정요구 대조' SKILL.md 뜯어보기

매번 반복하는 내 업무 절차도 파일 한 장으로 굳힌다

시정요구 조치결과 대조 — SKILL.md

언제 부르나

결산 검토 착수 시 · 정부 조치결과 보고 접수 시

절차

1. 전년도 시정요구 목록 확보 — 건별 문안 원문 그대로 ← NABOSTATS
2. 정부 조치결과 보고와 건별로 맞대기 (조치완료·조치중)
3. 문안이 비슷해도 이행이 같은지 — 제도 변경으로 확인
4. 유형 판정 — 이행 · 부분 이행 · 형식 이행 · 미이행
5. 해마다 되풀이되는 항목 표시 — 연례 반복

산출

대조표 + [확인 필요] 태그 — 인정 여부는 사람이

/시정요구-대조 2025회계연도 조치결과 대조해줘

- ◆ 한 마디로 **매번 같은 대조** — 담당·주니어가 불려도 누락 없이
- ◆ 절차가 **글(.md)**이라 과 표준으로 굳히고 개선·공유
- ◆ [확인 필요] 태그로 **미확정을 스스로 표시**한다 — 판정은 사람 몫

INSIGHT

2부 실습③(응용 갈래)에서 여러분이 만들 것이 바로 이런 파일 — 오늘 오후, 매일 반복하는 업무 하나를 골라 절차서로 굳힌다

비고 설명용 골격 — 3절 시연②의 전수 대조를 절차서로 옮긴 것. 다른 후보는 집행 부진 사업 선별 · 서면질의 회신 취합

명령어는 사람이 부르는 자동화, 훅은 조건이 부르는 자동화다

① 명령어 (Slash Command)

절차를 담은 md 한 장 = 새 명령어 하나 — 자주 쓰는 워크플로우의 단축키

/조사분석관 → 수집 → 정리 → 전년 대비 표 생성까지 한 줄

타이핑 몇 글자가 절차 전체를 부른다 — 팀원 누가 불러도 같은 결과

② 훅 (Hook)

파일 저장·명령 실행 전후에 **자동 개입** — 사람이 잊어도 규칙은 지켜진다

예: 원본 데이터 수정 시도는 자동 차단

예: 산출물 저장 때마다 수치 표기 규칙 검사

예: 작업 기록을 로그로 자동 추적

INSIGHT

검증을 의지가 아니라 구조에 맡기는 방법 — 훅의 예시가 전부 '검증 계열'인 것은 우연이 아니다

API는 '기계용 창구', MCP는 그 창구에 꽂는 '표준 플러그'

낯선 약어지만 하는 일은 단순하다 — AI에게 **지금 이 순간의 원자료**를 가져다주는 통로

API — 기관이 낸 기계용 창구

사람은 **눈으로 보고 옮겨 적는다** — API는 기계가 **곧바로 받아 가는 창구**

같은 자료, 창구만 다르다

MCP — 창구에 꽂는 표준 플러그

제각각인 창구 규격을 **AI 쪽에서 하나로 통일**한 어댑터
콘센트마다 다른 어댑터 → USB-C 하나로

내 질문

"환경부 집행을 뽑아줘"



MCP

표준 플러그로 연결
규격 통일



API

기관의 기계용 창구
열린재정·국회·법제처



오늘자 원자료

그 수치로 답한다

INSIGHT

이 통로가 없으면 AI는 **학습된 기억**으로 답한다 — 철 지난 숫자의 출처 — 붙이면 **오늘자 원자료**로 답한다

예결위 심사에만 쓰라고 만든 창구가 따로 있다

API — 소프트웨어끼리 데이터를 주고받는 공식 창구

자료원	무엇이 나오나
열린국회정보 예결위 전용 항목	예결산 심사정보 (회부·상정·의결 일자) · 예비심사 정보 — 상임위 예비심사 검토보고서 원문 주소 까지 나온다
의안정보시스템 예결산 심사연혁	연도별 정부안 → 증액 → 감액 → 국회확정 이 한 표로 · 예산안 첨부서류 일괄 내려받기
국가법령정보 법제처 공동활용	법령·시행령·훈령 조문을 조·항·호·목 단위 로 — 근거 조문을 원문 그대로

INSIGHT

기관이 **기계가 읽으라고 열어 둔 창구** — 인증키는 대개 무료이고, 신청 절차도 물어보면 알려 준다

재정·계약·통계 창구

심사의 **숫자와 사실**이 나오는 곳

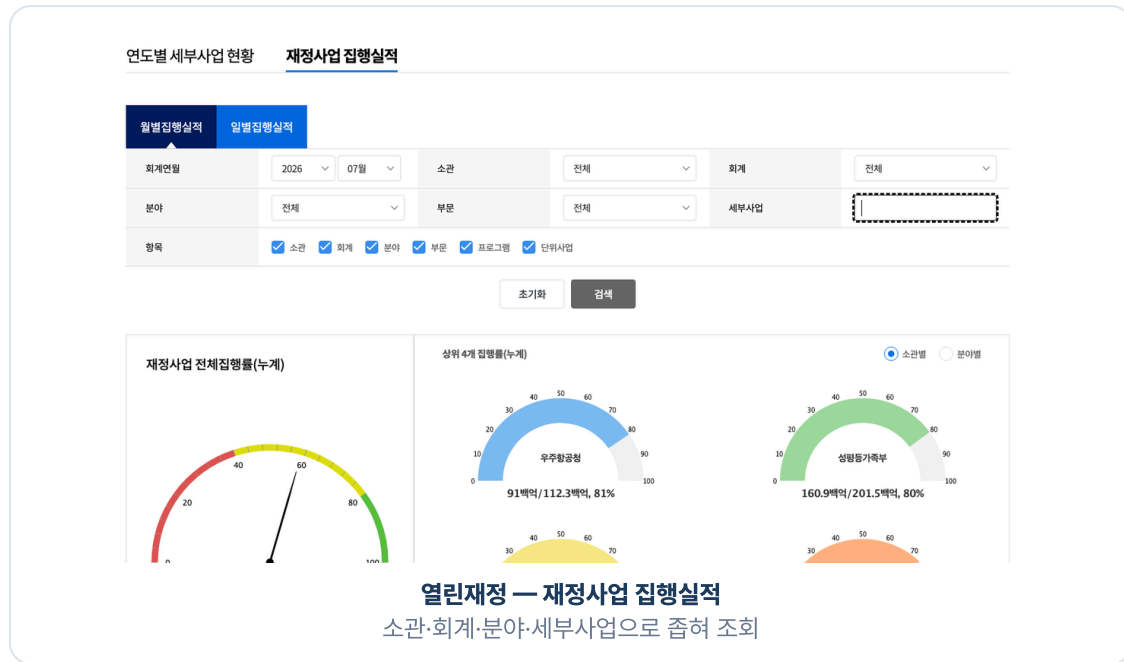
자료원	무엇이 나오나
열린재정 openfiscaldata.go.kr	세입·세출 예산과 결산 · 월별 집행실적 · 이월·불용 추이 · 국가채무 — 결산 심사의 본진
나라장터 조달청 · 조달데이터허브	낙찰·계약·계약변경이력 — 집행률은 100%인데 계약이 12월에 몰려 있는지 를 본다
KOSIS · 지표누리	거시지표·인구 — 예산안 전제 검증 ·수요 추계

INSIGHT

"〇〇부 불용 추이 뽑아서 전년 대비 표로" — 가 **실제로 도는 요청**이 된다

원료 창구는 이렇게 생겼다

앞 장의 표가 실제 화면으로는 이렇다 — 둘 다 로그인 없이 열린다



INSIGHT

화면에서 눈으로 보던 것을 **API로 받으면 그대로 표가 된다** — 같은 자료, 사람용 창구와 기계용 창구의 차이일 뿐



MCP는 AI의 USB-C — 공공 API엔 아직 연결기가 드물다

공개 창구를 **표준 규격 하나로** AI에 꽂는 연결 통로



한국 공공 API를 붙이는 길	어떻게	언제 고르나
① 직접 래퍼 제작	공식 API 키를 받아 얇은 래퍼를 만든다 — 하나에 한두 시간	기관 업무 · 만들면 팀 자산
② 범용 MCP 활용	웹 조회로 공개 페이지를 읽고 한글·PDF를 처리	키 발급 전에 당장
③ 공개 MCP 사용	korean-law-mcp 법령 검색·조문 대조 · kordoc 한글·PDF 파싱	이미 검증된 영역

CAUTION

기관 업무에는 **공식 API + 자체 래퍼**가 안전하다 — 공개 MCP는 시점 확인 필수

용어 **MCP**(Model Context Protocol) — AI와 외부 시스템을 잇는 표준 규격 · **래퍼** — 기존 API를 그 규격으로 감싸는 얇은 연결 프로그램

바로 붙는 MCP 도구 — 한글 문서·법령을 AI 손에 쥐여준다

오늘 바로 설치해 쓸 수 있는 검증된 공개 MCP들

도구 (MCP)	무엇을 하나	예결위라면
kordoc	.hwp·.hwp·PDF·엑셀 → Markdown 파싱 (표·수치 구조화)	부처 제출자료·전년 보고서를 AI가 읽게 — 수치 대사의 앞단
korean-law-mcp	법제처 법령·조문 검색 · 신·구 조문 대조	예비비·기금 요건의 근거 조문 확인 · 인용 조문을 현행 원문 과 대조
Tiro	회의·공청회 음성 → 회의록.md (화자별·요약)	공청회·간담회 녹취를 구조화된 회의록으로

```
# 설치도, 사용도 자연어로
"github.com/chrisryugj/kordoc MCP로 붙여줘" · "이 폴더 hwp 전부 마크다운으로 바꿔줘"
```

INSIGHT

원칙 — 원천은 분리하고, 파싱 결과는 검산 가능한 텍스트로 남긴다 — 원본 .hwp는 그대로 두고 AI는 변환본으로 일한다

비고 kordoc-korean-law-mcp 핸들 github.com/chrisryugj/kordoc · [SeoNaRu/korean-law-mcp](https://github.com/SeoNaRu/korean-law-mcp) · 열린재정·열린국회정보는 MCP가 아닌 **OpenAPI**라 자체 래퍼(앞 장)로 붙인다 · MCP 존재·유지보수는 시점 확인

요청 한마디가 부품 전부를 차례로 지낸다

따로 배운 부품들이 실제로는 **한 줄**로 이어져 돈다



- ◆ 규칙·절차는 **한 번 써 두면 자산** — 다음 요청에도 그대로 쓰인다
- ◆ 손발이 붙어야 오늘자 수치 — MCP·API가 없으면 학습된 기억으로 답한다
- ◆ 검증은 끝이 아니라 공정 안 — 혹은 조건이 부르고, 검증자는 독립으로 판정한다

INSIGHT

부품 하나하나가 아니라, 이 **한 줄 전체가 '판'(하네스)** — 같은 모델이라도 판이 다르면 결과가 갈린다

같은 모델, 같은 요청 — 공정이 다르면 오류가 0이 된다

같은 요청을 **순정 AI**와 **검증 팀**에 주고 인용을 전수 열람한 실측

감사 항목	순정 AI 단독	검증 팀 (킷)
수치·귀속 오류	2건 + 연도 혼입 1건 — 전량 확정 어조	0건
출처 부착률	60%	100% — 전 주장 인라인
작성 중 원문 열람	0회 — 검색 요약만으로 인용	핵심 수치 전건 확인 후 인용
과정 기록	본문 md 1개	설계·판정·근거·검수 파일 사슬
문장·구성 품질	우위 — 유려함·맥락 폭	골격은 명료, 문장은 담백

같은 모델이라도 '판'이 결과를 가른다

하네스 — 지금까지 본 부품을 하나로 엮은 작업 환경

- ◆ 비유 — 모델은 갓 입사한 **인재**, 하네스는 회사의 **제도**
직무기술서 · 결재선 · 품질검사 — 같은 인재도 제도가 좋아야 성과를 낸다
- ◆ 같은 AI라도 **무엇을 쥐여 주고 무엇을 검증시키느냐**가 품질을 가른다
- ◆ 1절의 "그럴듯한 오류"를 잡는 곳도 모델이 아니라 **판(검증 장치)**이다
- ◆ 방금 본 CLAUDE.md · 스킬 · 서브에이전트 · 검증자 = 전부 이 판의 **부품**
- ◆ **"자리를 비웠을 때 승부가 난다"** — 없으면 **실패도 밤새 쌓인다**

완료조건·자동검증·종단조건이 갖춰진 판이라야 맡길 수 있다

INSIGHT

모델 경쟁은 회사들의 몫 — 실무자의 경쟁력은 **판을 짜는 능력**에서 나온다

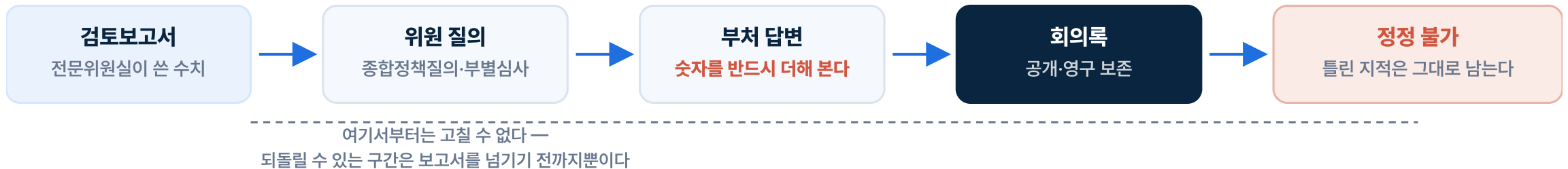
SECTION 5

틀리면 안 되는 것을 규칙으로 굳힌다

확인을 당부하는 대신, 확인을 건너뛸 수 없게 만든다

검토보고서의 수치는 회의록에 남는다

숫자는 보고서에서 끝나지 않는다 — **질의가 되고 회의록에 남는다**



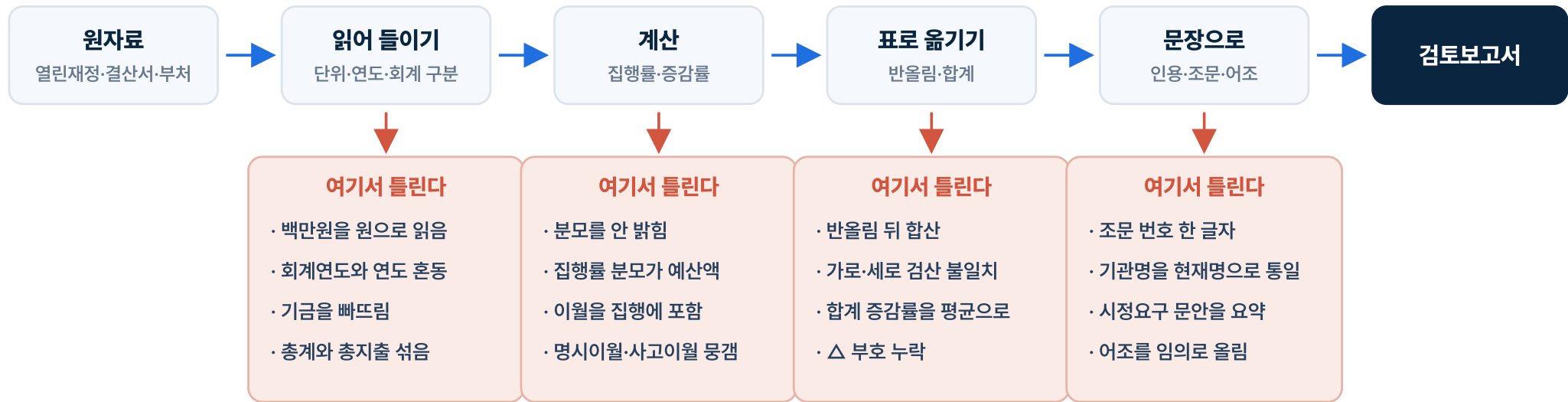
- ◆ 검토보고서는 **소위 심사의 기준선** — 틀린 수치는 심사를 왜곡한다
- ◆ 부처는 **자기 사업의 숫자를 반드시 검산한다**

CAUTION

그래서 예결위 업무에 AI를 들일 때 첫 질문은 "무엇을 시킬까"가 아니다 — **"어디서 틀릴 수 있나"가 먼저다**

수치 하나가 보고서에 실리기까지 지나는 길

틀림은 수치가 갈아타는 자리에서만 생긴다 — 그 자리를 먼저 지도로 만든다



INSIGHT

원자료에서 문장까지 갈아타는 자리마다 규칙을 하나씩 박아 두면 — 그 사이에서 AI가 자의적으로 고를 여지가 사라진다

수치를 넘기기 전에 되묻는 목록

앞 장의 지도를 물음으로 바꾼 것 — 채택 전에 이 목록을 통과시킨다

- ? **회계연도** — 결산연도·집행연도·예산연도 중 무엇인가
- ? **단위** — 백만원인가 천원인가 억원인가. 표 머리에 썼는가
- ? **분모** — 증감률의 분모는 전년 본예산인가 최종예산인가
- ? **예산현액** — 예산액 + 전년도이월 + 예비비 + 이·전용이 맞는가
- ? **집행률** — 분모가 예산현액인가. 이월을 집행에 넣지 않았는가
- ? **이월** — 명시이월·사고이월·계속비이월을 갈랐는가
- ? **회계** — 일반회계·특별회계·기금 중 무엇인가. 기금이 빠지지 않았는가
- ? **기준** — 총계인가 순계인가 총지출인가
- ? **분류** — 12대 분야인가 16대 분야인가. 세부사업 코드가 연도 간 같은가
- ? **검산** — 가로와 세로가 동시에 맞는가. 반올림 전 값으로 계산했는가
- ? **출처** — 확정결산인가 열린재정인가 부처 자료인가
- ? **잠정** — 열린재정 집행실적은 잠정치다. 조회 시점을 썼는가
- ? **인용** — 조문·시정요구 문안·감사원 지적을 원문과 대조했는가

INSIGHT

외워서 쓰는 목록이 아니다 — 파일에 박아 넣으면 되묻는 일 자체를 에이전트가 한다

AI의 '깔끔하게 정리하는' 본능이 사실을 무너뜨린다

일반적인 오류는 사람도 낸다 — 문제는 **사람은 하지 않는데 AI만 하는 종류**가 따로 있다는 것이다

AI가 하는 일	실제로 이렇게 망가진다	왜 사람 눈에 안 띄나
기관명을 통일 한다	2025년 집행 주체는 기획재정부 , 조치 대상은 재정경제부·기획예산처 — 한 문단 안에서 주체가 갈린다	누가 잘못했고 누가 고쳐야 하는가가 지워진다
△ 를 정리한다	△ 는 마이너스 — △7,084 의 기호를 떼면 흑자와 적자가 뒤집힌다	숫자는 그대로라 계산에 안 걸린다
단위를 맞춘다	표는 135,077 (백만원), 본문은 1,350억 7,700만원 — 옮겨 적다 1,000배가 어긋난다	표와 본문은 따로 읽혀 대조되지 않는다
조문을 줄인다	제26조제1항제5호가목6) 에서 한 글자가 틀리면 위반 판단의 근거가 소멸한다	형식이 그럴듯해 원문을 안 열어 본다

CAUTION

전부 "**더 보기 좋게 만들려는**" 행동 — 금지할 것은 오류가 아니라 **정리하려는 습관**

AI가 알고 있는 정부는 지난해 정부다

모델은 **배운 시점의 세상**을 안다 — **개편·개정** 뒤에도 옛 이름과 옛 조문을 내놓는다

AI가 그대로 쓰는 것	실제로는	그래서 무슨 일이 생기나
기획재정부	예산·성과관리는 기획예산처 , 결산·국고·세제는 재정경제부 로 갈렸다 통계는 국가데이터처	조치 요구 대상 기관이 틀린다 — 없는 부처에 시정을 요구하는 문장 이 된다
"국가재정법 제8조에 따른 재정사업 성과관리"	제8조는 2021년에 삭제됐다 — 성과관리는 제4장의2(제85조의2~제85조의12) 로 옮겨 갔다	근거 조문이 없는 지적이 된다 — 부처가 조문 하나로 반박하면 그 항목은 끝난다
기관 누리집 주소	한국재정정보원은 fis.kr — 널리 인용되는 다른 주소는 운영되지 않는다	각주의 링크가 죽어 있다 — 검증한 적 없다는 표시가 된다

CAUTION

웹에서 열어 보면 3분이면 드러난다 — 확신에 찬 어조라 열어 볼 생각이 안 들 뿐

부탁에서 규칙으로, 규칙에서 검산으로

"수치 조심해줘"는 매번 잊힌다 — 같은 내용을 파일에 적으면 매번 읽고, 코드로 적으면 어길 수 없다

위로 갈수록 —

잊을 수 없고, 어길 수 없다

말로 부탁한다

"수치 틀리지 않게 해줘"

대화를 새로 열면 사라진다

파일에 적는다

CLAUDE.md · SKILL.md

에이전트가 일할 때마다 읽는다

잊지 않는다 · 팀에 복사된다

다만 읽고도 어길 수는 있다

기계가 검산한다

합계·증감·단위를 코드로 재계산

어긋나면 작업을 멈추고 돌려보낸다

어길 수가 없다 —

통과하지 못하면 산출물이 나오지 않는다

산술은 LLM이 아니라 코드에 맡긴다

이것이 가장 자주 놓치는 한 줄이다

INSIGHT

규칙을 지키게 만드는 힘은 문장의 강도에서 나오지 않는다 — 그 규칙이 어디에 적혀 있는가에서 나온다

업무지침에 박아 두는 재정 규칙

앞의 목록을 에이전트가 매번 읽는 문장으로 적으면 이렇다

재정 수치 취급 규칙 (어길 수 없음)

1. 연도 "작년·올해·내년"을 쓰지 않는다. **2025회계연도** 결산처럼 항상 명시한다.
2. 단위 금액에는 단위를 값과 함께 옮긴다. 표 머리에 (단위: **백만원**)을 쓴다.
3. 분모 증감률에는 분모를 병기한다. 집행률의 분모는 **원자료의 예산현액** 칼럼을 쓴다.
4. 구분 회계 구분과 기준을 라벨로 붙인다. 이월은 명시·사고·계속비를 구분해 적는다.
5. 계산 가로와 세로를 **코드로** 재계산한다. 반올림은 표시할 때만 한다.
6. 표기 Δ 는 **마이너스다**. 기관명을 현재 이름으로 통일하지 않는다.
7. 인용 조문·시정요구 문안은 **원문 대조 전까지** 초안이다. 요약하지 않는다.
8. 보안 미공개 부처 제출자료·심사 전 내부자료는 입력하지 않는다.

INSIGHT

오늘 아무것도 안 해도 이 블록 하나는 복사해 갈 것 — 이것만으로 절반은 된다

합계가 어긋나면 작업이 멈춘다

산술은 읽어서 지키는 것이 아니라 — 다시 계산해서 확인하는 것

집필가가 표를 만들어 넘긴 직후

▶ 계산 게이트 실행 ... 표 4종 / 행 186

표1 부처별 세출 결산 가로 OK 세로 OK

표2 사업별 집행 현황 가로 OK 세로 불일치

└ 합계 2,481,006 ≠ 항목합 2,481,012

 차이 6 (백만원) — 반올림 후 합산 흔적

표3 이월·불용 내역 가로 불일치

└ 예산현액 - 지출액 - 이월액 ≠ 불용액

 행 27: 이월액이 집행액에 포함된 것으로 보임

표4 전년 대비 증감 분모 미표기

■ 게이트 불통과 — 산출물을 내보내지 않음

수정 요청을 작성해 집필가에게 반환합니다.

- ◆ 사람이 표를 더해 보기 전에 기계가 먼저 더해 본다
186행을 눈으로 계산하는 일은 일어나지 않는다
- ◆ 대개 반올림 후 합산과 이월을 집행에 포함에서 걸린다
- ◆ 통과하지 못하면 산출물이 나오지 않는다
경고를 띄우고 넘어가게 두면 경고는 읽히지 않는다 — 멈추는 것이 핵심
- ◆ 계산은 LLM이 아니라 코드가 한다
같은 모델에게 "다시 확인해줘"라고 하면 같은 실수를 반복한다

CAUTION

규칙 가운데 기계로 옮길 수 있는 것은 전부 옮긴다 — 사람의 주의력은 기계가 못 하는 판단에 남겨 둔다

수치가 서로 다를 때 무엇을 정보원으로 삼는가

수치가 어긋나는 일은 드물지 않다 — 미리 정해 두지 않으면 편한 쪽이 석인다



- ◆ 불일치 원인은 **기준시점 · 기준 · 기금 범위 · 사업 재분류**
원인 없이 '수치가 다름'이라고만 쓰면 답변 한 줄로 무력화된다
- ◆ 한쪽을 고르지 말고 — **둘 다 적고 원인을 쓴다**
- ◆ **조용히 한쪽을 고르는 것이** 가장 나쁘다
- ◆ 열린재정 집행실적은 **결산 검증 전 잠정치**
확정 결산에서 바뀌면 그 지적은 사후에 틀린 것이 된다

규칙은 기억이 아니라 파일에 산다

먼저 적는다

무엇을 시킬지보다 **무엇이 틀리면 안 되는지**를 먼저 적는다 — 급소 목록이 곧 업무지침의 초안이다

이 순서를 뒤집으면 규칙은 사고가 난 뒤에야 생긴다

파일이 된다

적은 것은 **CLAUDE.md**와 **절차서**가 된다 — 에이전트는 일할 때마다 읽고, 사람은 잊어도 파일은 잊지 않는다

과 단위로 복사하면 그대로 과의 표준

기계가 지킨다

기계로 옮길 수 있는 규칙은 **검산 게이트**로 내린다 — 통과하지 못하면 산출물이 나오지 않는다

사람의 주의력은 판단에 남겨 둔다

INSIGHT

오늘 이 절에서 가져갈 한 문장 — **AI가 만든 수치와 인용은, 원문과 대조하기 전까지 전부 초안이다**

보안 경계를 먼저 긋는다 — 넣지 않을 것부터 정한다

넣는다 — 공개 창구의 자료

열린재정 공표치 · 의안정보 공개문서 · 법령 조문
공개된 검토보고서·회의록 · 공개 통계 · 감사결과 공개문
재현 가능하고, 출처를 밝힐 수 있고, 외부에 나가도 되는 것

넣지 않는다

미공개 부처 제출자료 · 심사 전 내부 검토자료
확정 전 회의 내용 · 위원 질의 초안 · 개인정보
프롬프트와 첨부파일은 외부 서버로 전송된다

◆ 경계를 긋고 나면 **공개 자료만으로 할 수 있는 일이 생각보다 많다**

◆ 기관 도입은 단계적으로 — **개인 학습 → 팀 시험 운영 → 보안 심의·가이드라인 → 확산**

개인 단계에서 충분히 검증한 뒤 기관 절차를 밟는 것이 정석

◆ 비용 구조 — 구독 Pro \$20 · Max \$100~200/월, API는 쓴 만큼

개인 학습·오늘 실습 수준은 Pro로 충분

◆ API 키는 환경변수로 분리 보관 — 코드·문서에 직접 쓰지 않는다

CAUTION

경계는 시작을 막는 이유가 아니라 — **공개 자료 범위 안에서 오늘 시작하는 이유다**

도구가 아니라 분석관의 판단이 산출물의 책임을 진다

검증 의무

숫자·인용·법령은 **원문 재확인**이 기본값 — '그럴듯함'과 '정확함'을 혼동하지 않는다

기록 보존

프롬프트·중간 산출물 보존 — **재현 가능성**과 감사 대응 — 킷의 산출물 사슬이 이를 자동화한다

귀속

AI 활용 여부는 기관 지침에 따라 표기 — **서명은 사람이 한다**

보안

민감 정보는 처음부터 분리 — 시작 전에 경계를 설정한다

INSIGHT

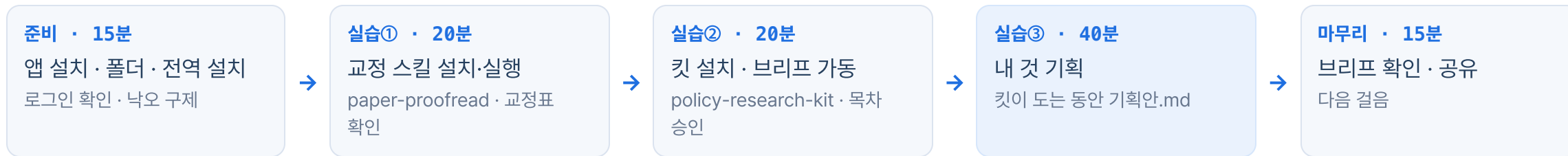
도구는 바뀌어도 **책임은 옮겨 가지 않는다** — 검증 · 기록 · 귀속 · 보안이 신뢰받는 사용자의 조건이다

PART 2 · HANDS-ON

실습 — 앱 설치 · 스킴 · 킷 · 내 것

데스크톱 앱 하나로 — 프롬프트를 복사해 붙이면 설치하는 앱이 한다

실습이 끝나면, 각자 '내 업무용 AI 팀' 기획안을 갖고 나간다



- ◆ **프롬프트는 외우지 않는다** — 화면의 **복사** 버튼으로 가져가 앱 대화창에 붙인다 · 터미널은 열지 않는다
- ◆ 실습마다 **확인 방법**이 정해져 있다 — "파일이 생겼는가"로 본다 · 말로 복창시키지 않는다
- ◆ 막히면 손 들기 — 설치 문제는 **준비 구간에서 전부 해결**하고 넘어간다 · 그래도 안 되면 옆자리와 짝 실습

INSIGHT

목표는 '완성'이 아니라 **돌아가는 초안** — 킷이 도는 동안 기획안을 쓰면 두 가지가 한 번에 남는다

데스크톱 앱을 깔고 로그인한다 — 터미널은 안 쓴다

① 내려받기

claude.com/download
Mac · Windows 데스크톱 앱



② 설치 · 로그인

계정으로 로그인
한 번만 하면 유지된다



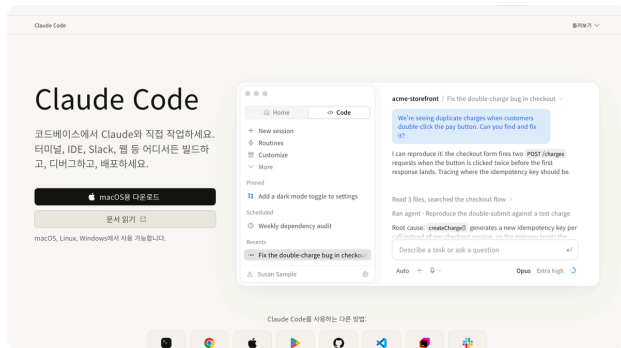
③ Code 탭 열기

앱 왼쪽의 Claude Code
여기가 오늘의 작업 화면



④ 확인

대화창에 한글로 한 줄
답이 오면 준비 끝



내려받기 페이지 — claude.com/download

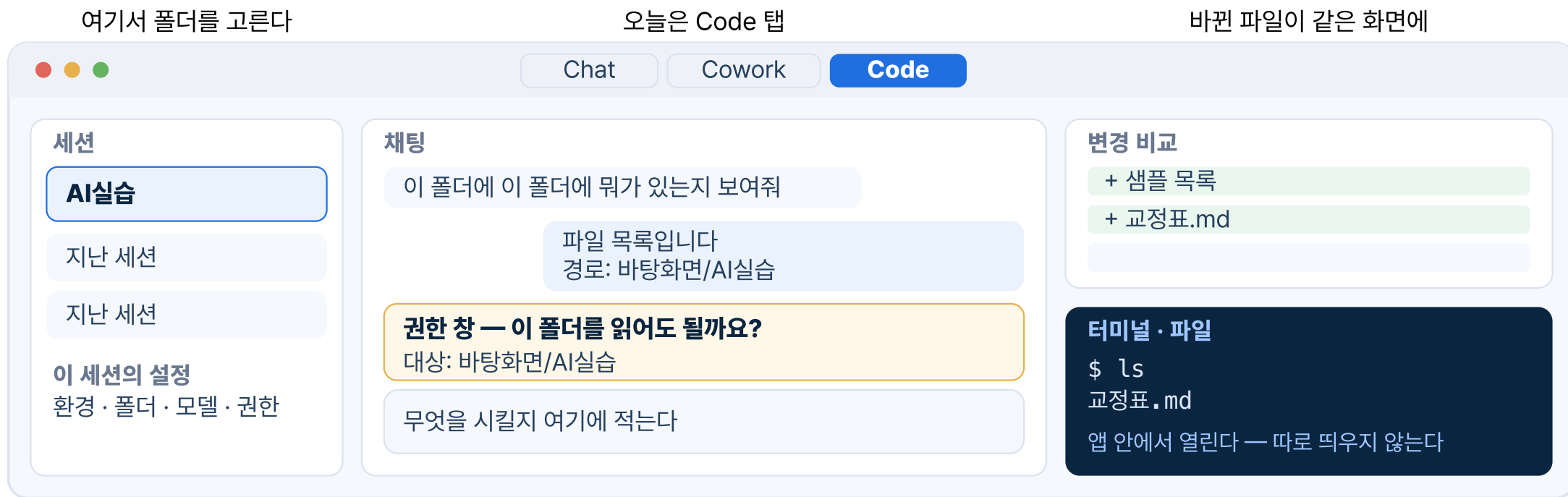
- ◆ **Windows**는 Git for Windows가 먼저
git-scm.com — 설치 후 앱을 다시 켜다 · 스킵 · 키트 설치가
이것을 쓴다
- ◆ **Mac**은 그대로 진행
처음 실행 때 개발자 도구 설치 창이 뜨면 '설치'
- ◆ 구독은 **Pro**면 오늘 실습에 충분

INSIGHT

확인 — 대화창에 "안녕, 지금 어떤 폴더에서 일하고 있어?" 라고 쳐 본다 · 경로가 답으로 오면 준비 끝

Code 탭의 구성 — 어디를 보고 어디에 적는가

오늘 쓰는 화면은 왼쪽 세션 · 가운데 대화 · 오른쪽 변경 비교 셋이 전부다

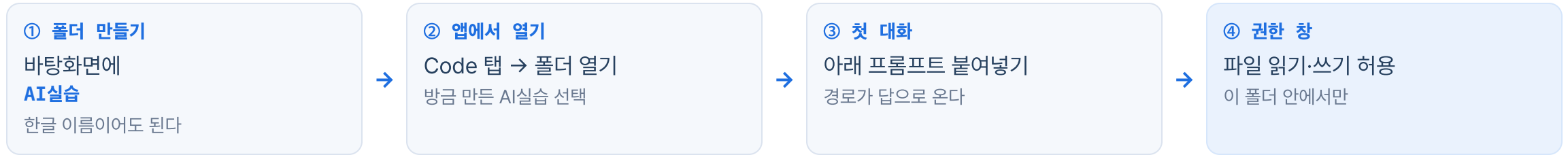


Claude 데스크톱 앱 Code 탭 — 강의용 구성도(배치는 앱 버전에 따라 다를 수 있다)

INSIGHT

가운데 노란 권한 창이 오늘 가장 자주 보게 될 것 — 대상 경로를 읽고 허용한다 · 작업 폴더 밖을 가리키면 거부

폴더 하나를 만들고 앱에서 연다 — 여기가 오늘의 작업실이다



지금 어떤 폴더에서 일하고 있는지 전체 경로를 알려주고, 이 폴더 안에 무엇이 있는지 보여줘.

- ◆ 답에 **AI실습** 경로가 보이면 성공 — 다른 폴더가 보이면 ②를 다시 한다
- ◆ 권한을 묻는 창이 뜨면 **허용** — 앱은 이 폴더 밖을 건드리지 않는다
오늘 만들어지는 파일은 전부 이 폴더에 쌓인다

INSIGHT

폴더가 곧 작업실 — 킷이 낸 브리프도, 내 기획안도 이 폴더에 생긴다 · 스킵·킷 자체는 **전역(~/.claude/)**에 설치해 어느 폴더에서나 쓴다

쓰기 전에 — 어떤 '두뇌'를 쓸지 · 바로 할지 '계획부터' 할지

필수는 아니지만 알면 편하다 — 작업에 맞는 **모델**을 고르고, 큰 일은 **계획 모드**로 방향부터 잡는다

모델 — 작업에 맞는 두뇌

성능·속도가 다른 두뇌를 **앱에서 골라 쓴다**

고성능(Opus) 설계·검증 · 균형(Sonnet)·빠름(Haiku) 단순·대량

계획 모드 (Plan)

계획을 먼저 보여주고 **승인하면 실행**

방향이 틀린 채 달리는 것을 막는다 — 목차 승인 관문과 같은 원리

INSIGHT

방향은 사람이, 실행은 AI가 — 큰 작업은 계획 모드로 먼저 확인한다

가장 작은 부품부터 — 스킬 하나를 설치하면 무엇이 달라지나

같은 원고에 같은 말을 해도 — **스킬이 있고 없**고가 결과를 가른다

"이 원고 교정해줘"	스킬 없이	paper-proofread 스킬로
기준	그때그때 AI가 정한다 — 부를 때마다 다르다	규칙 파일 (번역투 14유형 · 학술 문장 규범)을 매번 같이 읽는다
산출	고친 문장만 채팅으로	교정표 — 심각도 · 위치 · 원문 · 수정 · 근거 (파일을 주면 수정본까지)
확인	무엇을 왜 고쳤는지 알 수 없다	표를 보고 채택·반려 를 사람이 결정한다

INSIGHT

스킬 = 절차 한 장(.md) — 4절에서 본 그 부품을 **지금 내 PC에 넣는다** · 설치 1분, 문장 하나로 실행 1분

설치 프롬프트 하나 — 붙여넣으면 앱이 저장소를 받아 온다

<https://github.com/parkjui92/paper-proofread> 저장소를 ~/.claude/skills/paper-proofread 폴더에 클론해줘.

- ◆ 권한 창이 뜨면 **허용** — 앱이 저장소를 받아 그 폴더에 넣는다
명령을 외울 일은 없다 · 무엇이 실행됐는지만 보면 된다
- ◆ 확인 — 그 폴더에 **SKILL.md**가 생겼으면 설치 끝
이 파일이 4절에서 본 '절차서' 그 자체다
- ◆ **~/.claude/skills/**는 **전역(글로벌)** — 어느 폴더를 열어도 쓸 수 있다
프로젝트 폴더 안 .claude/skills/에 두면 그 폴더에서만 보인다 — 오늘은 전부 전역에 설치한다
- ◆ 스킬은 **대화를 시작할 때 읽힌다** — 설치 후 **새 대화**를 열어야 스킬로 인식된다

INSIGHT

~/.claude/skills/ 아래에 폴더 하나가 생긴 것 — 그것이 설치의 전부다 · 지우면 삭제, 복사하면 공유, 전역이라 어디서나 호출

새 대화를 열고 — 틀린 문장 하나를 스킬로 교정해 본다

paper-proofread 스킬로 다음 문장을 교정교열해줘. 교정표(원문·수정·근거)도 같이 보여줘.

아버지가방에들어가시는지곰곰히생각해봤다. 이 정책은 향후 지속가능한 발전을 위해 검토되어질 필요성이 있다고 사료된다.

- ◆ 확인 — 교정표에 **띄어쓰기**(아버지가 방에 들어가시는지) · **맞춤법**(곰곰히→곰곰이) · **이중 피동**(검토되어질→검토될) · **번역투**(필요성이 있다고 사료된다)가 잡히면 성공
- ◆ 스킬 없이 같은 문장을 넣어 보면 차이가 보인다 — **근거 열**이 있고 없고
- ◆ 파일로도 된다 — 원고를 대화창에 **끌어다 놓거나**, "**다운로드 폴더의 OO.docx 교정교열해줘**"처럼 경로를 말하면 수정본과 교정표를 **파일로** 낸다
.docx · .hwp · .pdf · .md — 실무에서는 이쪽 · 오늘은 안내만

INSIGHT

"AI에게 **절차를 주면** 그대로 일한다" — 프롬프트를 매번 길게 쓰지 않는다 · 이것이 실습②·③의 뼈대다

실습 ① 체크아웃 — 셋 다 되면 다음으로

- 1 ~/.claude/skills/paper-proofread/SKILL.md 가 있다 — "paper-proofread 스킬이 설치돼 있는지 확인해줘"로 물어도 된다
- 2 교정표가 나왔다 — **곰곰히→곰곰이 · 검토되어질→검토될**이 근거와 함께 보인다
- 3 파일로 시키는 법을 안다 — **끌어다 놓기** 또는 **경로 말하기** (오늘은 안내만)

증상	처방 — 그대로 말하면 된다
"git을 찾을 수 없다"	Windows Git 미설치 — git-scm.com에서 설치 후 앱 재시작
스킬을 모르는 듯 그냥 교정만 한다	새 대화를 열지 않은 것 — 새 대화 에서 다시 · 그래도면 "~/.claude/skills/paper-proofread/SKILL.md"를 읽고 그 절차대로 해줘"
다른 폴더를 열었더니 스킬이 없다	프로젝트 폴더 안(.claude/skills/)에 깔린 것 — ~/.claude/skills/ (전역)로 옮겨 달라고 한다
권한 창이 계속 뜬다	'이 세션 동안 허용'을 고른다 — 폴더 밖 작업은 거절해도 된다

프롬프트 하나면 팀이 내 PC에 들어온다

<https://github.com/parkjui92/policy-research-kit> 저장소를 클론하고 폴더 구조를 보여줘. 그다음 skills 폴더의 폴더들을 ~/.claude/skills/ 로, agents 폴더의 .md 파일들을 ~/.claude/agents/ 로 복사해줘.

- ◆ 확인 — 표에 **오케스트레이터 · 설계 · 조사 · 집필 · 검수 · 한글 변환** 스킬과 **팀원 에이전트**(설계자·조사관·작성자·검수자·변환가)가 보이면 성공
- ◆ 4절에서 본 그 팀이 **지금 내 PC에** 들어와 있다 — 여기부터는 여러분 것
- ◆ 설치 후 **새 대화**를 연다 — 스킬·에이전트는 대화를 시작할 때 읽힌다

INSIGHT

스킬도 에이전트도 결국 **md 파일** — **전역(~/.claude/)**에 두면 어느 폴더를 열어도 앱이 읽는다 · 어느 PC에서나 같은 방법

설치된 팀의 취업규칙을 열어 본다 — 검수 기준부터

~/ .claude/

```
|— skills/
|   |— rnd-policy-research-orchestrator/ ← 조율자
|   |— policy-research-design/ ← 설계
|   |— policy-research/ ← 조사
|   |— policy-report-writing/ ← 집필
|   |— policy-report-review/ ← 검수
|   |— rnd-hwp-x-export/ ← 한글 변환
|— agents/
|   |— policy-research-designer.md
|   |— policy-research-investigator.md
|   |— policy-report-writer.md
|   |— policy-report-reviewer.md ← 검수자
|   |— hwp-x-exporter.md
```

~/ .claude/agents/policy-report-reviewer.md 를 열어서, 검수자가 초안을 어떤 축으로 검수하는지 항목만 뽑아 보여줘.

◆ 확인 — 검수 축 **논리 · 근거 · 타당성 · 정량성 · 출처 실재**가 보인다

◆ 적어 보기 — "**우리 보고서라면 어떤 축을 하나 더 두겠는가?**"

수치 대사 · 법령 인용 · 시정요구 대조 — 실습③의 재료가 된다

INSIGHT

블랙박스가 아니다 — **읽을 수 있으면 고칠 수 있다** · '도구를 쓴다'와 '팀의 규칙을 고친다'의 차이가 여기서 갈린다

새 대화에서 — 내 담당 분야의 공개 현안으로 브리프를 시킨다

□□ 분야의 최근 현안을 주제로 4쪽짜리 정책브리프를 작성해줘.

- ◆ □□에는 **본인 담당 분야** — 품질을 스스로 감정할 수 있어야 한다
예: 지방재정 · 산업기술 R&D · 보건복지 재정 · 국방 조달
- ◆ 확인 — 화면에 "**스킬 실행 중**" · "**에이전트 실행 중**" 표시가 떠야 킷이 실제로 도는 것
그냥 글을 쓰기 시작하면 스킬이 안 읽힌 것 — 새 대화에서 다시
- ◆ "정책브리프"라는 말이 조율자를 깨운다 — 시작하면 **설계자가 먼저 목차**를 가져온다 → 다음 장
- ◆ 규칙 — **미공개·내부 자료는 넣지 않는다** (1부 보안 원칙)

INSIGHT

브리프 모드(4~8쪽)는 **20~40분**이면 완주한다 — 표준 보고서(50쪽+)는 같은 파이프라인의 긴 버전일 뿐

목차 승인 관문에서 사람이 방향을 잡는다

설계 검토를 통과한 목차가 오면 — 승인 전에 **말로 고칠 수 있다** — 앞서 본 관문을 직접 겪는 순간이다

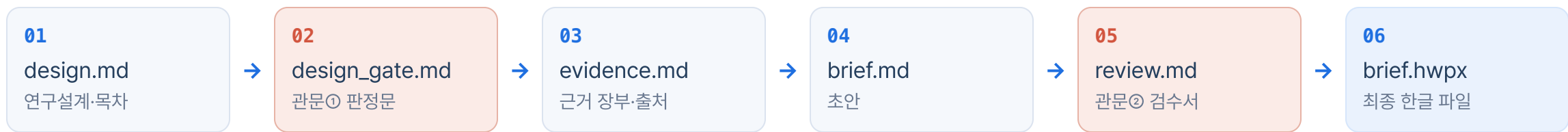


- ◆ "3장을 **해외 사례 중심**으로 바꿔줘"
- ◆ "결론에 정책 제언을 **우선순위 순**으로 정리해줘"
- ◆ 승인하면 조사·집필이 시작된다 — **이때부터 실습③(내 것 기획)으로 넘어간다** · 킷은 그 대화에서 계속 돈다

INSIGHT

승인 전에는 조사·집필이 진행되지 않는다 — **이 멈춤이 품질 장치다** — 승인 후엔 조사→집필→검수→한글 변환까지 자동

승인 뒤에는 기다리지 않는다 — 폴더에 쌓이는 것만 가끔 본다



증상	처방 — 그대로 말하면 된다
목차 없이 그냥 글을 쓰기 시작한다	스킬이 안 읽힌 것 — 새 대화에서 다시 · 프롬프트에 "정책브리프"가 들어 있는지 확인
권한 창(파일·웹 검색)이 뜬다	허용 — 조사관이 공개 자료를 검색하는 단계다
멈춘 것 같다	"지금 어느 단계까지 진행됐어? 폴더에 생긴 파일을 보여줘"
한글(.hwp) 변환에서 오류	04·05까지 있으면 실습은 성공 — 변환은 "brief.md를 다시 hwp로 변환해줘"로 재시도

INSIGHT

체크아웃 — 02 판정문과 05 검수서가 생겼으면 1부에서 본 설계 관문과 검수 관문을 내 주제로 겪은 것이다

킷이 도는 동안 — 내 업무용 에이전트의 기획안을 쓴다

오늘 만드는 것은 코드가 아니라 **기획안.md** 한 장 — 이것이 있어야 개발이 시작된다

① 재료

반복 업무 하나 고르기
다음 장의 여섯 칸



② 프롬프트

[] 채워 새 대화에 붙이기
복사 버튼 · 9항목 기획안



③ 저장

기획안.md가 폴더에
모호한 곳은 AI가 되묻는다



④ 시작

"기획서 읽고 이대로 만들어줘"
오늘은 여기까지 가면 대성공

- ◆ 킷이 도는 대화는 **그대로 두고** — **새 대화**를 하나 더 열어 기획한다 (같은 폴더)
- ◆ 한 스킬 = 한 업무 — '**다 해주는 비서**'는 **실패한다** · 첫 버전은 딱 하나만
- ◆ 씨앗 예시 — **시정요구 대조 · 서면질의 회신 취합 · 집행 부진 사업 선별 · 사업설명자료 요약**

INSIGHT

기획안은 **파일로** 남긴다 — 채팅은 사라지지만 파일은 다음 주에도 그대로 개발 명세가 된다

기획 전에 — 반복 업무 하나를 골라 여섯 칸을 채운다

- ◆ ① 한 스킬 = 한 업무 — '다 해주는 비서'는 실패한다
- ◆ ② 지금 방식을 숫자로 — 누가 · 얼마나 자주 · 몇 시간
- ◆ ③ 입력은 실물로 지목 — '관련 자료'가 아니라 **파일 이름**으로 짚는다
- ◆ ④ 절차는 신입에게 인수인계하듯
사람에게 설명하지 못하는 업무는 에이전트도 하지 못한다
- ◆ ⑤ 출력 형태 확정 — 표 양식 · 목차 · 종결 어미까지
- ◆ ⑥ 틀리면 안 되는 것 — 5절의 급소 목록에서 **내 업무에 해당하는 줄**을 골라 적는다

INSIGHT

씨앗 예시 — 시정요구 대조 · 서면질의 취합 · 집행 부진 선별 · 사업설명자료 요약

기획안(개발명세) 작성 TIP

- ◆ 1. 개요 — 한 줄 정의 · 타겟 · 문제 · 성공 기준
- ◆ 2. 범위 — 첫 버전 포함/제외 (제외 이유까지)
- ◆ 3. 핵심 시나리오 — 입력 → 처리 → 출력을 구체 예시로
- ◆ 4. 예상 기술 병목 — 항목별 문제 / 대응 방안
- ◆ 5. 구성요소 — CLAUDE.md · 스킬 · MCP 목록
- ◆ 6. 하네스 — 폴더 · 권한 · 피드백 루프 · 실패 처리
- ◆ 7. 워크플로우 — 전체 흐름 · 역할·입출력 표
- ◆ 8. 결과물 대시보드 — 무엇을 어떤 화면에
- ◆ 9. 구현 순서 — 무엇부터 만들지 단계별 체크리스트

INSIGHT

앞은 "무엇을·왜", 뒤는 "어떻게·검증" — 하네스 설계가 혼자 도는 에이전트를 만든다

이 한 덩어리를 복사해 []만 채우고 붙인다

나는 Claude Code로 [만들고 싶은 AI 에이전트]를 만들려고 해. 같이 기획안을 짜자.
△ 코드 구현은 하지 마. 이번 산출물은 기획안 md 파일 1개뿐이야. 모호한 곳은 먼저 나에게 물어봐.

[배경]

- 누구를 위한 거냐면: [나 자신이어도 OK] · 왜 만드냐면: [지금 반복하는 작업 / 풀고 싶은 불편]
- 이렇게 되면 성공이야: [무엇을 대신해주면 성공인지]
- 첫 버전은 [딱 이 작업 하나]만 자동화돼도 좋아 - 뭘 뺄지는 네가 제안해줘.
- 할 일 상세: [내 분야의 특성과 노하우, 작업 순서를 듬뿍]
- 입출력 예시: [이런 입력(파일·질문·데이터)] → [이런 결과물] · 제약: [보안 · 비공개 데이터 · 예산 · 시간]

[기획안은 아래 9항목으로 써 줘]

- 1 개요(한 줄 정의·타겟·문제·성공 기준) 2 범위(첫 버전 포함/제외 - 제외 이유까지)
- 3 핵심 시나리오(입력→처리→출력을 구체 예시로) 4 예상 기술 병목(항목별 문제/대응 방안)
- 5 구성요소 설계(CLAUDE.md 규칙 초안 · 스킬 목록[이름/역할/트리거] · MCP[이름/용도/이유])
- 6 하네스 설계(폴더 구조 · 도구·권한[금지작업] · 피드백 루프 · 실패 처리 · 테스트 3개 이상)
- 7 워크플로우·서브에이전트(전체 흐름 mermaid · 역할/입력/출력/도구 표)
- 8 결과물 대시보드(무엇을 어떤 화면에) 9 구현 순서(단계별 체크리스트)

다 되면 기획안.md 파일로 저장해줘.

- ◆ 킷이 도는 대화는 그대로 두고 **새 대화**를 열어 붙인다 — [] 안은 앞 장의 여섯 칸에서 모호한 곳은 AI가 먼저 물어 온다 — 답하면서 기획안이 채워진다

더 가기 — 파일 두 곳만 고치면 검토보고서 팀이 된다

바꿀 곳	이렇게 시킨다 — 그대로 말하면 된다
보고서 목차 골격 작성자의 절차서	"표준 목차를 사업 개요 → 예산·집행 현황 → 검토 쟁점 → 조치 요구 구조로 바꿔줘. 종결은 개조식으로 하고 '~할 필요가 있음'을 기본형으로 써줘"
검수 기준 검수자의 업무기술서	"검수 측에 5절 급소 목록을 넣어줘 — 회계연도·단위·분모·집행률·이월 구분·회계 구분·검산·출처·인용을 전부 확인하게 하고, 하나라도 확인 못 하면 통과시키지 말라 고 적어줘"
◆ 5절에서 본 재정 규칙 블록을 작업 폴더의 <code>CLAUDE.md</code>에 그대로 붙여 넣는다 복사만 하면 되므로 가장 먼저 할 일이다 — 이후의 모든 작업이 이 규칙 위에서 돈다 ◆ 고친 뒤 브리프를 다시 돌려 보면 — 목차와 검수 리포트가 달라져 있다 개조 전후를 비교하는 것이 이 갈래의 학습 완성 ◆ 잘 된 개조는 결국 파일이므로 — 과 단위로 복사해 쓰면 그대로 과의 표준이 된다	

비고 목차 체계는 소관마다 다르다 — 본인이 쓰던 최근 보고서의 목차를 그대로 불러 주는 편이 빠르다

더 해 볼 사람만 — 오늘자 법령을 가져오는 두 갈래

5절에서 본 **지난해 정부** 문제를 손으로 푸는 실습 — 법령 API로 부르거나, 법령 MCP를 쫓는다

선택 ① API — 주소로 부른다

법제처에서 **내 인증키**를 받아 파일에 두고, 말로 시켜 오늘자 조문을 가져온다

인증키 발급 5분 · 무료·즉시 · 한 번 받으면 계속 쓴다

선택 ② MCP — 대화에 쫓는다

법령 도구를 설치해 매번 주소를 치지 않고 **대화 안에서** 조문을 받는다
설치 한 번 · 인증키는 ①에서 받은 것을 그대로 쓴다

- ◆ 순서는 ① 다음 ② — MCP도 같은 인증키를 쓰므로 ①을 건너뛰면 ②에서 막힌다
- ◆ 망분리·보안 정책으로 막히면 여기서 멈춘다 — **기관 정책 확인 사항** · 화면 시연으로 대신 본다

INSIGHT

여기까지 하면 — AI가 **기억**으로 답하던 조문을 **오늘자 원문**으로 답하게 바꾼 것이다

선택 ① 1단계 — 내 인증키 받기

법제처 국가법령정보 공동활용은 **무료·즉시** — 가입 이메일의 @ 앞부분이 곧 인증값이다

- 1 **open.law.go.kr** 접속 → 오른쪽 위 **로그인** → 아래 **사용자 가입**(이메일)
- 2 상단 **OPEN API** → 왼쪽 **OPEN API 신청**
활용 목적 "업무 참고·학습용" · 사이트 URL "없음"으로 적어도 된다
- 3 승인 뒤 **마이페이지** → **API인증키관리**에서 확인
OC 값 = 가입 이메일의 @ 앞부분 — 예: hong@korea.kr이면 hong

INSIGHT

공용 시연키를 쓰지 않는다 — 남의 키로 성공하면 자기 키를 끝내 발급하지 않고 실무에서 같은 자리에 막힌다

출처: [법제처 국가법령정보 공동활용 — 회원가입 · OPEN API 신청](#)

선택 ① 2단계 — 키는 파일에, 일은 말로

먼저 — 파일은 앱이 만들고, 값은 내가 직접 넣는다

이 폴더에 .env 파일을 만들고 LAW_OC= 까지만 적어줘. 값은 내가 직접 넣을 테니 비워 두고, 파일을 열어서 보여줘.

값을 넣은 뒤 — 주소는 내가 조립하지 않는다

이 폴더 .env의 LAW_OC 값을 읽어서, 법제처 국가법령정보 API로 "국가재정법"을 검색해줘. 법령명·법령ID·공포일자·시행일자·소관부처를 표로 정리하고, 네가 실제로 호출한 주소도 같이 보여줘.

- ◆ 확인 — 표에 **시행일자**가 있고, 소관부처가 **기획예산처·재정경제부**로 나오면 오늘자를 받은 것
기획재정부로 나오면 원자료가 아니라 기억에서 답한 것이다
- ◆ 대조 — 보여 준 **주소를 브라우저에 붙여넣어** 같은 값이 오는지 본다
이 한 번이 "가져왔다"와 "지어냈다"를 가른다
- ◆ **채팅에 키 값을 붙여넣지 않는다** — 대화 기록·화면 공유·캡처에 그대로 남는다 · "값을 보여줘"도 하지 않는다

출처: [법제처 OPEN API 활용가이드 — 현행법령 목록 조회](#)

선택 ② — 법령 MCP를 대화에 쏙는다

주소를 주고 — 설치는 맡긴다

<https://github.com/SeoNaRu/korean-law-mcp> 저장소를 클로드 코드 MCP로 설치해줘. README에 적힌 설치 방법을 그대로 따르고, 인증키는 이 폴더 .env의 LAW_OC 값을 쓰게 해줘. 끝나면 무엇이 설치됐는지 알려줘.

설치 뒤 — 연결 확인

/mcp

- ◆ 확인할 것 — 이름이 목록에 있고 · 상태가 **connected**이고 · 도구 이름이 보이면 성공
하나라도 없으면 설치가 안 끝난 것 — 새 대화를 열고 다시 본다
- ◆ "설치를 활성화하라"는 안내가 보이면 **/reload-plugins** 한 줄
Node.js가 없다는 오류면 nodejs.org에서 설치 후 앱 재시작
- ◆ 같은 방법으로 **kordoc**(한글·PDF 읽기)도 붙는다 — 주소만 바꾸면 된다

법령 MCP가 없을 때와 있을 때

같은 문장을 그대로 넣어도 — **도구가 있느냐 없느냐**에 따라 돌아오는 것이 다르다

“국가재정법 제1조 목적을 찾아줘”

MCP가 없을 때

학습해 둔 기억에서 문장을 만든다
또는 웹에서 요약본을 찾아 옮긴다

- 조문이 **옛 판본**일 수 있다
- 시행일자가 불지 **않는다**
- 출처가 요약본 · 블로그일 수 있다

맞는지 확인하려면 내가 법제처에 다시 들어가야 한다

MCP가 있을 때

get_law_text 도구를 부른다
→ 법제처 오픈 API 응답

- **오늘자** **현행** 조문이 온다
- 시행일자 **20260911**이 함께 온다
- 어느 주소를 불렀는지 남는다

확인이 대화 안에서 끝난다 — 이것이 MCP를 까는 이유

MCP는 새 능력을 주는 것이 아니라, 매번 주소를 치던 일을 대화 안으로 들여놓는 것이다

INSIGHT

5절의 **"AI가 아는 정부는 지난해 정부"**를 푸는 방법 — 기억에서 꺼내지 말고 **오늘자 원문**을 부르게 한다

작게 만들고, 돌려 보고, 검증하며 키운다

- 1  자연어로 말하듯 — "코드 짜줘"라고 시키는 대신 "**이런 일을 하고 싶어**"라고 말한다 — 목적·제약·기대결과를 문장으로
- 2  한 번에 완성 말고 반복으로 — 첫 결과물은 초안. 대화 이어가며 조각내 개선
- 3  작은 샘플로 먼저 검증 — 파일 100개 돌리기 전에 3개부터. 예상대로 되는지 보고 확장
- 4  검증은 필수, 할루시네이션은 상수 — 숫자·인용·법령은 원문 재확인. **그럴듯함 ≠ 정확함**
- 5  민감정보는 처음부터 분리 — 미공개 자료·개인정보는 시작부터 제외, API 키는 환경변수로

완성된 시스템을 설계하려 들기보다 — 간단한 것부터 만들고, 쓰면서 키운다

오늘 쓴 규칙과 검산기를 공개해 두었다

5절에서 본 것들은 설명용 예시가 아니다 — 내려받아 그대로 쓰는 물건으로 만들어 공개했다

검산 게이트

표의 **합계** · **관계식** · **증감** · **분모** · **△ 표기** · **자릿수**를 기계로 본다 — 통과 못 하면 내보내지 않는다

파이썬 표준 라이브러리만 — 설치도 인증키도 필요 없다 — 마크다운 표와 HTML 표를 읽는다

재정 규칙 블록

작업 폴더 `CLAUDE.md`에 붙여 넣는 업무지침
이 블록 하나는 꼭 가져가기

검토보고서 문체 규격

골격 · 계층 번호 · 개조식 종결 · **어조 사다리** · 고정 양식 표
결산 검토보고 3,810쪽을 전수 계량해 굳힌 규격

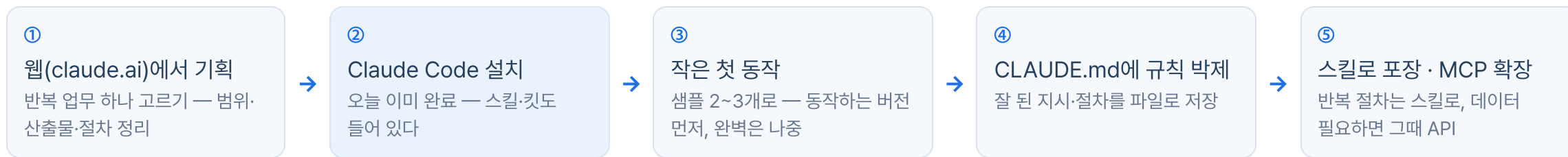
INSIGHT

자료원 지도도 함께 — 조회 경로와 **자료요구로 받아야 하는 것의 경계까지**

조치결과 역검증 절차

자기보고를 **근거로 되짚는** 절차 — 이행 · 부분 이행 · **형식 이행** · 미이행

내일 아침, 이 다섯 계단으로 시작한다



◆ 오늘 여러분은 ②를 마쳤고 ③~⑤를 맛봤다 — 내일은 ①부터 **내 업무로**

◆ 확산 경로 — 개인 학습 → 과 단위 공유 → 기관 절차

오늘 만든 것을 계속 키워 가자

박주이

한양대학교 과학기술정책연구소 연구교수

parkjui92@gmail.com

설치·개조 관련 후속 질문은 메일로도 받음

오늘의 자료

github.com/parkjui92

정책연구 킷 · 교정 스킴 · 오늘 쓴 도구 일체 — 계속
공개 유지

도구 모음 참고

MCP · API 등 실무 도구 카탈로그 —
예결산·의안·법령 API, kordoc 등

[참고자료 열기](#) ↗

무엇을 언제 부르는지 한눈에

INSIGHT

질문 환영 — 특히 "우리 과 이 업무에도 되나?"류의 질문이 오늘 가장 좋은 질문